# V-OS

Un sistema operativo gráfico para el Amstrad CPC 6128

Manual

Hecho a partir de docs/manual/es/*.md y sdk/README.es.md con tools/mkdocs.py (make docs).

# Índice

V-OS es un sistema operativo gráfico para el Amstrad CPC 6128, escrito desde cero en ensamblador Z80. Tiene:

- una barra de menús, ventanas que se pueden arrastrar, diálogos y botones, que se usan con el teclado o con un ratón AMX
- tres vistas para los archivos y los programas (Escritorio, Programas, Commander) y los ajustes (Ajustes)
- seis idiomas como complementos: inglés, griego, francés, español, alemán e italiano
- programas que funcionan uno al lado del otro y continúan donde se quedaron, un accesorio de escritorio y programas en segundo plano
- Studio, un editor de bloques al estilo de Scratch que funciona en el propio CPC
- una API con 42 llamadas y un SDK para programas en ensamblador

Este manual también está en griego: 00-index.md, en inglés: 00-index.md, en francés: 00-index.md, en alemán: 00-index.md y en italiano: 00-index.md.


- 1. Primeros pasos
- 2. Las vistas
- 3. Programas
- 4. Idiomas
- 5. Studio: programas con bloques
- 6. Bloques en el PC: el editor web y `vbc.py`
- 7. El SDK de V-OS: programas en ensamblador
- 8. Desarrollo de V-OS
- 9. En un CPC real

# 1. Primeros pasos

## Qué se necesita

- Un Amstrad CPC 6128 o un emulador. V-OS se prueba en Caprice32.
- El disquete de V-OS en la unidad A:, es decir, el `build/vos.dsk` que genera `make`, y el disquete de los programas, `build/apps.dsk`, en B:.
- De forma opcional:
  - un ratón AMX
  - una segunda unidad (B:)
  - una ampliación de memoria (p. ej. 512K), para más programas a la vez

## El arranque

Desde el BASIC:

```
RUN"VOS
```

V-OS carga el núcleo, desactiva las ROM y toma el control de toda la máquina. Detecta por sí solo la ampliación de memoria.

- **Primera vista:** el Escritorio. Se puede cambiar en Ajustes, en la línea VISTA INICIAL.
- **Idioma:** inglés. Si hay un `SETUP.CFG` en el disquete, arranca con el idioma y los ajustes guardados en él.
- **Salir:** ARCHIVO > SALIR AL BASIC (CONTROL+Q en las vistas). Pide confirmación y reinicia el CPC.

## La pantalla

V-OS usa toda la pantalla (overscan): 384×272 píxeles en modo 1 (4 colores) o 192×272 en modo 0 (16 colores).

- **Barra de menús** (arriba): los menús de la vista o del programa, normalmente ARCHIVO o el menú del programa, VER y AYUDA. A la derecha está el idioma del teclado (EN, EΛ, ...).
- **Línea inferior:** en cada vista, las teclas que funcionan en ella.

## Teclas que funcionan en todas partes

| Tecla | Qué hace |
|---|---|
| F1 | Escritorio |
| F2 | Programas |
| F3 | Commander |
| F4 | Ajustes |
| F5 | abre y cierra el accesorio de escritorio (ACCESORIO) |
| F6 | los programas en marcha, para elegir uno |
| ESC | abre el primer menú. En los menús, ←/→ cambian de menú, ↑/↓ eligen una opción, RETURN la ejecuta, ESC cierra |
| CONTROL+letra | los atajos de los menús (p. ej. ^Q, ^L) |

| | |
|---|---|
| CONTROL+L | cambio de idioma (VER > CAMBIAR DE IDIOMA) |
| CONTROL+K | el diálogo con las teclas (AYUDA > TECLAS) |
| CONTROL+SHIFT | cambia el teclado: inglés, el del paquete y, con TECLADOS: LOS SEIS en Setup, los de los otros idiomas |
| TAB, SHIFT+TAB | mueven el foco entre los botones. RETURN, ENTER o SPACE pulsan el botón que tiene el foco |

F1–F6 son las teclas f1–f6 del teclado numérico del CPC.

## El puntero y el ratón

- **Con un ratón AMX:** el puntero se mueve con el ratón, y su botón hace clic. El ratón se activa y se configura en Ajustes (RATÓN AMX, LECTURA AMX).
- **Sin ratón:** el puntero se mueve con las flechas, y COPY hace clic. En las vistas que usan las flechas para elegir (Escritorio, Programas, Commander), las flechas eligen archivos y programas en lugar de mover el puntero.

Casi todo funciona de las dos maneras: un clic en un archivo o programa lo selecciona, y un segundo clic lo abre.

## Ventanas y diálogos

- **Arrastrar:** todas las ventanas se pueden arrastrar por su barra de título: los diálogos, la ventana ARCHIVOS del Escritorio, el accesorio de escritorio y las ventanas de los programas de bloques. Mientras se mantiene pulsado el botón se ve un contorno discontinuo, y la ventana se vuelve a dibujar donde se suelta.
- **Diálogos:** esperan una respuesta. TAB y ←/→ cambian de botón, RETURN lo pulsa, y ESC elige el último (normalmente NO o CANCELAR).
- **Introducción de texto** (p. ej. al renombrar): se escribe al final, DEL borra, RETURN acepta y ESC cancela. También se puede escribir en el idioma del paquete.

## Disquetes y archivos

- **Formato:** V-OS lee y escribe disquetes AMSDOS (formato DATA o SYSTEM) en A: y B:. El BASIC también puede leer sus archivos.
- **Nombres:** 8 caracteres más 3 para el tipo (`NOMBRE.TIP`).

Tipos que se encontrará:

| Tipo | Qué es |
|---|---|
| `.BIN` | un programa de V-OS (o un archivo binario) |
| `.VBN` | un programa de V-OS hecho con bloques |
| `.VBS` | un programa de Studio (su código fuente) |
| `.OVL`, `.MSG`, `.LNG` | partes del propio V-OS: vistas, textos, idiomas |
| `.SYS` | archivos del sistema (p. ej. `VBRUN.SYS`, el reproductor de Studio) |
| `SETUP.CFG` | los ajustes |

Los ajustes, los idiomas y las vistas se leen de A:, así que el disquete de V-OS debe quedarse ahí.

## Mensajes de error

Los errores de disco se muestran en un diálogo: UNIDAD NO LISTA, DISCO PROTEGIDO, NO SE ENCUENTRA, DISCO LLENO, DIRECTORIO LLENO, EL NOMBRE YA EXISTE, NOMBRE NO VÁLIDO y otros.

Dos más tienen que ver con los programas:

- **NO ES PROGRAMA V-OS:** el archivo no tiene la cabecera de un programa.
- **CIERRE PRIMERO OTRO PROGRAMA:** no hay sitio. Véase el capítulo 3.

# 2. Las vistas

V-OS tiene cuatro "vistas": maneras de ver los archivos y los programas, y los ajustes. Se cambia de vista con F1–F4 o desde el menú VER. Cada vista recuerda qué archivo estaba seleccionado, y la siguiente selecciona el mismo archivo.

## Los menús comunes

Los dos últimos menús son iguales en todas las vistas y en los programas:

| **VER** | | **AYUDA** | |
|---|---|---|---|
| ESCRITORIO | F1 | TECLAS | ^K |
| PROGRAMAS | F2 | ACERCA DE V-OS | |
| COMMANDER | F3 | | |
| AJUSTES | F4 | | |
| PROGRAMAS (en marcha) | F6 | | |
| CAMBIAR DE IDIOMA | ^L | | |

AYUDA > ACERCA DE V-OS muestra la versión (0.15 beta; cada hito suma 0.01), la fecha de compilación, el autor (Revive8bit, 2026) y el arquitecto (Vasper).

## Escritorio, F1

Modo 1. Una ventana ARCHIVOS muestra los archivos de una unidad como iconos, cuatro por fila. El tipo va escrito dentro del icono y el nombre debajo, y un `.VBN` lleva al lado una marca de "play". A la derecha, dos iconos eligen A: o B:. La barra de título muestra el formato del disquete y el número de archivos, y el espacio libre se muestra abajo.

| **Tecla** | **Qué hace** |
|---|---|
| flechas | eligen un archivo (↑/↓ una fila) |
| TAB | la siguiente unidad: A:, B:, DISK (con una tarjeta M4) |
| RETURN, ENTER | abrir: un programa se ejecuta, un archivo de un tipo conocido se abre en su programa, los demás muestran su información |
| ^C | copiar a la siguiente unidad (desde DISK: a A:) |
| ^D | borrar (pregunta antes) |
| ^N | renombrar |
| ^R | releer los discos |
| ratón | clic en un archivo: lo selecciona, otra vez: lo abre. Clic en una unidad: la abre |

- **Menú ARCHIVO:** ABRIR, INFO DEL ARCHIVO (^I), COPIAR A LA SIGUIENTE, BORRAR, RENOMBRAR, RELEER LOS DISCOS, GUARDAR EL ESCRITORIO (^S), SALIR AL BASIC.
- **GUARDAR EL ESCRITORIO** (^S) escribe la ventana de archivos tal como está (dónde, su tamaño, su unidad, o cerrada) en `A:SETUP.CFG`: en el siguiente arranque el Escritorio la abre así enseguida. El ^S de Setup también la guarda.
- **La ventana ARCHIVOS** se puede arrastrar por su título y se queda ahí, incluso al cambiar de vista.
- **Al principio no hay ventana ni unidad elegida:** se ve el fondo (y su imagen). Una unidad (un clic en su icono, o TAB) abre su ventana; la X de su título la cierra, igual que un clic en la unidad abierta. El cuadradito de la esquina inferior derecha agranda o reduce la ventana en

iconos enteros (2 a 5 por fila, 1 a 5 filas; cuatro por cinco al principio), y la barra de la derecha muestra qué parte de la lista se ve: un clic encima o debajo de su parte oscura desplaza una ventana. El tamaño se guarda, como el sitio.

- **Abrir según el tipo** (también en el Commander): los `.TXT` se abren en NOTEPAD, los `.MD` en MARKDOWN, los `.PIC` y `.LYR` en PAINT, los `.VDB` en DATABASE, los `.VSH` en SHEET, los `.MUS` en MUSIC, que empieza a tocarlos. El programa se busca en la unidad del archivo, luego en A: y luego en B:. Si no se encuentra, aparece "NINGÚN PROGRAMA PARA ESTE ARCHIVO". Si el programa tiene cambios sin guardar, pregunta primero.

## Programas, F2

Modo 0, 16 colores. Muestra solo los programas de una unidad (`.BIN`, `.VBN`, `.BAS`), al principio A:, como baldosas en una cuadrícula de 3×3, con las páginas siguientes debajo. La línea de página empieza con la unidad (`A:`, `B:`). Cada programa que tiene icono lo muestra en su baldosa. La baldosa seleccionada es blanca.

| Tecla | Qué hace |
|---|---|
| flechas | eligen (↑/↓ una fila. Más allá del borde, la página siguiente o la anterior) |
| RETURN, ENTER | ejecuta el programa. Si ya está en marcha, continúa donde estaba |
| ^A | lo carga como accesorio de escritorio (COMO ACCESORIO). Después F5 lo abre |
| ^R | releer el disco |
| TAB | la siguiente unidad (UNIDAD SIGUIENTE): A:, B:, DISK. Un programa iniciado desde B: vuelve a B: |
| ratón | clic: selecciona, otra vez: ejecuta |

**Menú ARCHIVO:** EJECUTAR, COMO ACCESORIO, UNIDAD SIGUIENTE, RELEER LOS DISCOS, SALIR AL BASIC.

## Commander, F3

Modo 1, dos paneles con listas de archivos: A: a la izquierda, B: a la derecha. Cada panel muestra el formato y el número de archivos en su título, y el espacio libre abajo. El panel activo tiene el título oscuro y una selección.

| Tecla | Qué hace |
|---|---|
| ↑/↓ | eligen (con SHIFT: una página) |
| TAB, ←/→ | el otro panel |
| RETURN, ENTER | abrir, como en el Escritorio. La información (tamaño, tipo, dirección de carga, los primeros bytes) con ^I |
| ^C | copiar al otro panel, de cualquier tamaño. Si hay un archivo con el mismo nombre, pregunta antes |
| ^D | borrar |
| ^N | renombrar |
| ^R | releer los dos discos |
| ^G | la siguiente unidad para el panel activo: A:, B:, DISK (también con un clic en su título) |
| ^F | buscar archivos: un nombre con `*` y `?` (`*.TXT`, `README.*`, `NOTE?AD.BIN`; sin punto, cualquier extensión), que se busca en A:, B: y DISK (la carpeta donde está). El panel activo lista lo encontrado, hasta 64 archivos, cada uno con su unidad (`B:README.TXT`). RETURN va al archivo en su unidad; ^C, ^D, ^N y ^V funcionan sobre los archivos encontrados tal como están; ^R o ^G termina la lista |

| | |
|---|---|
| ^V | ver el texto: las primeras líneas del archivo elegido en el otro panel, hasta que se vuelva a dibujar. Una imagen (`.SCR`, como las guardan PAINT y el CHART de SHEET) se ve en toda la pantalla en modo 0 con sus propias tintas; una tecla o un clic vuelve al Commander |
| ^E | copiar el disco: todo el disco del panel activo en el del otro panel (A: y B:), pista por pista, cada pista formateada antes como la del origen (DATA o SYSTEM). Pregunta antes: los archivos del otro disco se pierden |
| ratón | clic: selecciona, otra vez: abre |

**Menú ARCHIVO:** ABRIR, INFO DEL ARCHIVO (^I), VER EL TEXTO (^V), BUSCAR (^F), COPIAR EL DISCO (^E), COPIAR AL OTRO PANEL, BORRAR, RENOMBRAR, RELEER LOS DISCOS, UNIDAD SIGUIENTE (^G), FORMATEAR DISCO (DATA), SALIR AL BASIC.

> FORMATEAR borra todos los archivos del disquete del panel activo. Pregunta antes.

## DISK: la tarjeta SD de la M4

Con una tarjeta M4, el Escritorio tiene un tercer icono de unidad, DISK, debajo de A: y B:, y TAB pasa de A: a B: y a DISK. También llegan a ella Programas y los paneles del Commander (^G). El título muestra la carpeta donde está, p. ej. `DISK:/GAMES/`.

- **Las carpetas** se ven como carpetas (un icono con pestaña, o DIR en el Commander). RETURN sobre una entra en ella, y `..`, la primera entrada, sube. En Programas también aparecen, para llegar a los programas que contienen.
- **Todo como en un disquete:** los programas se ejecutan desde ella, los archivos se abren en sus programas, y ^C, ^D y ^N copian, borran y renombran. Las copias entre DISK y un disquete llegan hasta 16K. Carpetas y archivos tienen nombres como en el CPC (8.3: `NOMBRE.EXT`); los nombres más largos no aparecen.
- **M4.SYS** (en `vos.dsk`) se carga desde A: la primera vez que se usa DISK.
- **La hora y las fechas:** la barra de menús muestra la hora de la M4 («09:25», a la izquierda del idioma del teclado) en cada vista y programa donde cabe, e INFO DEL ARCHIVO (^I) da la fecha y la hora de un archivo de la tarjeta bajo su tamaño. La hora viene del servidor de hora de la M4 (puesto con `NETSET` en el CPC).
- **En los programas:** OTHER DRIVE (^W en SHEET) pasa de A: a B: y a DISK, y la línea de abajo dice `DISK:NOMBRE.TXT`. La lista de OPEN muestra también las carpetas: RETURN en una (o en `..`) muestra esa carpeta, y SAVE escribe en la carpeta desde donde abrió. READER escribe la tarjeta como `C:` en sus direcciones.
- **Las bases de datos** van también en DISK: el ^W de DATABASE llega allí, y STREETS ejecutado desde la tarjeta guarda allí su STREETS.VDB.
- **Aún no:** la tarjeta SD no se formatea desde V-OS. V-OS sigue arrancando desde un disquete en A: (escriba `|DISC` antes de `RUN"VOS"`: con una tarjeta M4, el CPC arranca en la tarjeta SD).

## Ajustes, F4

Modo 1. Una línea para cada ajuste. Los cambios se aplican al momento. Al guardarlos (RETURN o ^S) se escriben en `A:SETUP.CFG` y valen también en el siguiente arranque.

| Ajuste | Valores |
|---|---|
| VISTA INICIAL | ESCRITORIO, PROGRAMAS, COMMANDER |
| IDIOMA | ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO |
| RETARDO REPETICIÓN | cuánto tiempo se mantiene pulsada una tecla antes de que empiece a repetirse: CORTO, MEDIO, LARGO |

| | |
|---|---|
| VELOC. REPETICIÓN | LENTA, MEDIO, RÁPIDA |
| RATÓN AMX | SÍ, NO |
| LECTURA AMX | NIVELES (al principio) o PULSOS: cómo se lee el ratón. NIVELES cuenta cada lectura con movimiento, PULSOS solo el comienzo de los pulsos. Si el puntero se mueve demasiado rápido o demasiado despacio, pruebe el otro |
| COLOR DEL FONDO | AZUL, NEGRO, VERDE, ROJO |
| TECLADOS | EN + IDIOMA, LOS SEIS (los que recorre CONTROL+SHIFT) |
| TEMA | CLARO, OSCURO (barras, menús, diálogos, ventanas y listas en negro con texto blanco; en los programas, sus menús y diálogos) |
| IMAGEN DE FONDO | NINGUNA, o una imagen `.SCR`: IZQ./DER. pasan por NINGUNA, A:, B: y DISK, y en una unidad se listan sus `.SCR` para elegir uno. El Escritorio la muestra en el centro del fondo, con el color del fondo alrededor; una imagen de PAINT (modo 0, 16 colores) en los cuatro colores del Escritorio. Necesita una ampliación de memoria (su último banco de 16K guarda la imagen); sin ella: NECESITA UNA AMPLIACIÓN DE MEMORIA. Tras tres minutos sin hacer nada en el Escritorio, la imagen se muestra sola (un salvapantallas); cualquier tecla o el ratón devuelven el Escritorio |

| Tecla | Qué hace |
|---|---|
| ↑/↓ | eligen un ajuste |
| ←/→ | cambian el valor (también un clic en la línea) |
| RETURN, ^S | guardar |

Al cambiar el IDIOMA se carga al momento el paquete de idioma desde A:. Véase el capítulo 4.

# 3. Programas

## Empezar y terminar

Un programa se inicia de una de estas maneras:

- desde Programas (F2): se selecciona y se pulsa RETURN, o se hace doble clic
- desde el Escritorio o el Commander: ARCHIVO > EJECUTAR
- desde otro programa, como Studio, que ejecuta lo que usted haga

Cada programa tiene su propio menú junto a los comunes VER y AYUDA. Termina desde su menú (normalmente BACK o STOP, CONTROL+Q) y vuelve a la vista desde la que se inició.

## Varios programas a la vez

Los programas de V-OS son de dos clases:

- **Pequeños, con tabla de reubicación:** DEMO, MIXED y CLOCK, los ejemplos del editor web (PONG, CATCH, STARCAT, COUNTER), y lo que haga el SDK. Comparten 12K de memoria, y funcionan juntos tantos como quepan.
- **Programas que necesitan toda la memoria:** STUDIO y lo que hace Studio (CALC, MINES, FLAP, TICKER y los suyos). También MARKDOWN y READER, que pasan de 12K. Sin ampliación de memoria funcionan solos. Si ya hay uno en marcha, aparece "CIERRE PRIMERO OTRO PROGRAMA".

Cuando se va a una vista (F1–F4) mientras un programa está en marcha:

- Un programa pequeño se queda en memoria. Se retoma desde Programas (RETURN sobre él) o con F6, y continúa exactamente donde estaba.
- Un programa que tiene toda la memoria termina, a menos que haya una ampliación de memoria.

**F6** (VER > PROGRAMAS) muestra la lista de los programas en marcha. Se elige uno con las flechas y RETURN. Si no hay ninguno en marcha, aparece "NINGÚN PROGRAMA EN MARCHA".

**Con una ampliación de memoria** (p. ej. 512K, que V-OS encuentra por sí solo al arrancar), cada programa que necesita toda la memoria recibe su propio banco. Así, STUDIO, CALC, DEMO y PONG pueden funcionar todos juntos. Desde M43 cada programa recibe también su propio banco de datos de 16K cuando lo necesita: lo que guarda allí no se pierde mientras corren otros programas o vistas.

## En segundo plano

Un programa que está detrás de otro, o detrás de una vista, normalmente **se congela** y continúa cuando vuelve al frente.

Algunos programas piden **seguir funcionando** también en segundo plano, p. ej. TICKER, que cuenta el tiempo. Mientras están ahí no dibujan ni suenan. Cuando vuelven al frente, también han contado el tiempo que estuvieron fuera.

En Studio esto se elige para su programa con STUDIO > IN BACKGROUND.

MUSIC es la excepción: su canción suena desde la interrupción, 50 veces por segundo, así que sigue sin cortes detrás de todo. Solo se para un momento mientras se lee el disquete.

## Ventanas

Con una **ampliación de memoria**, CLOCK, SYSINFO, DIARY, TELNET, WEBGET y DATABASE se abren en una **ventana** sobre el escritorio, encima de las ventanas de los otros programas en marcha. Lo mismo un programa de Studio en modo 1 que abre una ventana propia (como CALC): la ventana muestra su interior, con sus botones. DIARY, TELNET, WEBGET y DATABASE muestran en la ventana su pantalla como era; si la reduces, muestra la parte de arriba a la izquierda. NOTEPAD, MARKDOWN, READER y SHEET también se abren en una ventana y disponen en ella su texto: tantas columnas y filas como caben, la línea de estado debajo, y de nuevo cada vez que cambias su tamaño. Su página (^M), y el gráfico de SHEET, se muestran a pantalla completa; ^M otra vez devuelve la ventana a donde estaba, del mismo tamaño. Cada ventana guarda una copia de sí misma en la memoria, así que al mover una ventana sobre las otras, estas vuelven enseguida, sin que sus programas las dibujen de nuevo. Bajo las ventanas se ve la imagen del escritorio (BACKGROUND en Setup). La ventana de archivos y los iconos de las unidades del escritorio también están entre las ventanas: se ven detrás de los programas, y en el escritorio (F1) las ventanas de los programas se ven sobre su superficie. Un clic en una ventana la trae delante, y un clic en la ventana o los iconos del escritorio trae el escritorio. La X de una ventana pide a su programa que se cierre como ^Q, así que NOTEPAD, por ejemplo, pregunta antes por los cambios sin guardar. Un programa en una ventana no cambia los colores de la pantalla. TELNET en una ventana sigue recibiendo cuando está detrás, si ninguna otra ventana lo tapa. El accesorio (F5) y la lista de programas (F6) se abren sobre las ventanas. Los demás programas se abren aún a pantalla completa.

- La barra de menús queda arriba y es la de la ventana **de delante**. Un clic en otra ventana la trae delante, con su barra.
- Arrastra el **título** de una ventana para moverla, y su **esquina inferior derecha** para cambiar su tamaño. El programa dispone lo que muestra para el nuevo tamaño.
- El **+** del título, o **F7**, da al programa toda la pantalla (bajo la barra), y de vuelta a una ventana. **X** cierra el programa.
- El reloj **sigue en marcha** mientras otra ventana está delante, siempre que ninguna lo tape.

## El accesorio de escritorio (ACCESORIO)

Un programa pequeño, como CLOCK, puede abrirse en una ventana **encima de cualquier cosa**: una vista o un programa.

1. En Programas, seleccione CLOCK y elija ARCHIVO > COMO ACCESORIO (^A).
2. A partir de entonces, **F5** lo abre y lo cierra, esté donde esté. ESC también lo cierra.

Mientras está abierto, recibe las teclas y los clics. Su ventana se puede arrastrar por su título. Cuando se cierra, se vuelve a dibujar lo que había debajo.

## La página (modo 2)

NOTEPAD, MARKDOWN, READER y SHEET tienen PAGE (MODE 2) en su menú, ^M. La pantalla pasa al modo 2: texto negro sobre papel blanco, 768 píxeles de ancho, con los menús y diálogos también negros sobre blanco. Los editores (NOTEPAD y MARKDOWN) muestran 80 columnas entre los bordes de la página, una línea donde termina cada página de 60 líneas, y PAGE n en la última línea. La vista previa de MARKDOWN y READER muestran la página en 80 columnas con sus estilos. SHEET muestra 10 columnas (A–J) con líneas entre ellas. ^M de nuevo vuelve al modo 1.

## Los programas del disquete

Un disquete del CPC tiene 180K, así que V-OS viene en dos. `vos.dsk` (A:) tiene el sistema, DEMO, MIXED, STUDIO y CLOCK, y las fuentes `.VBS` de los programas VB, que Studio abre desde A:. `apps.dsk` (B:) tiene los demás, de PONG a WEBGET: en el Launcher, TAB muestra B:. Algunos programas tienen su propio disquete, como dice la tabla.

| Programa | Qué es |
|---|---|
| **STUDIO** | el editor de bloques: se hacen programas en el CPC (capítulo 5) |
| **DEMO** | el primer programa de V-OS: un cuadrado que se mueve solo. Las flechas lo dirigen y SPACE cambia su color |
| **MIXED** | la "pantalla mixta": una franja de 16 colores (modo 0) entre franjas de modo 1. Un clic muestra las coordenadas |
| **CLOCK** | el tiempo desde que arrancó V-OS; con una tarjeta M4, la hora real y, como programa, la fecha (del servidor de hora de la M4, ajustado con `NETSET` en el CPC). También funciona como accesorio de escritorio |
| **CALC** | una calculadora en una ventana con botones, hecha en Studio (también existe `CALC.VBS`) |
| **MINES** | un buscaminas de 8×6 con 8 minas, hecho en Studio. Un clic abre una casilla, y NEW empieza una partida nueva |
| **BRICKS** | romper ladrillos: una pala con ←/→, 32 ladrillos, 3 vidas, niveles con una pelota cada vez más rápida. En GAME OVER, SPACE para una partida nueva |
| **VROUVES** | tower defence, del juego de CPC LET'S GO FOR VROUVES: 7 pantallas, del Kafenio al cementerio, con 10 oleadas cada una. Está en su propio disquete, `vrouves.dsk`: en B:, y TAB en el Launcher. Las flechas mueven el cursor, 1–6 elige un defensor, SPACE construye, S vende, N trae la oleada siguiente, P hace pausa. Puntos por cada invasor y por los aldeanos que quedan al final de una pantalla. El disquete guarda `VROUVES.SAV`: CONTINUE empieza desde la pantalla alcanzada, HIGH SCORES muestra las cinco mejores con sus nombres. Escrito con el SDK (`myapps/vrouves`) |
| **NOTEPAD** | un editor de texto en modo 1, 48×22, para archivos de hasta 4K, o de hasta 16K con una ampliación de memoria: entonces guarda unos 2,5K alrededor del cursor en su memoria y el resto en su propio banco de datos, y SELECT ALL toma esa parte. NEW, OPEN (solo archivos `.TXT`), SAVE, SAVE AS, OTHER DRIVE. Flechas, DEL, CLR y RETURN; un clic coloca el cursor. La línea de abajo muestra la línea y la columna. Menú EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT y las flechas seleccionan. El portapapeles (hasta 1K) es común a todos los programas. PRINT ^P imprime el texto en una impresora. UNDO ^Z (menú EDIT) deshace el último cambio: lo que se escribió o pegó, o lo que quitaron DEL, CLR, CUT o escribir sobre una selección (hasta 256 caracteres); ^Z otra vez lo rehace. FIND ^F (menú EDIT) busca un texto desde el cursor, en todo el texto (A–Z sin mayúsculas), y lo selecciona; FIND AGAIN ^G el siguiente; REPLACE ^R pide el texto y lo que va en su lugar, y lo cambia en todo el texto (NOT FOUND si no está) (`myapps/notepad`) |
| **PAINT** | dibujo en modo 0 con 16 colores: una imagen de toda la pantalla del CPC, 160×200 sin el overscan (`.SCR`, 16K, que también cargan el BASIC y otros programas del CPC; sin ampliación de memoria 128×128, `.PIC`), o de 64×64 con zoom 2× en 3 capas (`.LYR`, NEW LAYERS). Herramientas: P lápiz, B pincel, L línea, X rectángulo, F relleno, C círculo (desde su centro hasta donde se suelta el botón); TAB cambia el color. Las flechas y COPY dibujan sin ratón. La ventana muestra 128×128 de la imagen: las barras a su derecha y debajo muestran qué parte, y un clic o un arrastre sobre ellas la mueve. Bajo los colores: dónde está el puntero en la imagen, y VIEW, la esquina superior izquierda de la ventana. Capas: 1–3 eligen una, H la oculta o la muestra, U y D la suben y la bajan, E es la goma. UNDO ^Z deshace la última herramienta, y ^Z otra vez la recupera (un relleno de toda la pantalla no se puede deshacer; después de que se ejecute otro programa no hay nada que deshacer). EXPORT .SCR (para la imagen de 128×128 y las capas; toda la pantalla ya es un `.SCR`): la imagen (las capas tal como se ven) como pantalla del CPC, un `NOMBRE.SCR` de 16K (modo 0, en el centro, con la paleta), para BASIC u otros programas. En BASIC: `MODE 0:LOAD"NAME.SCR",&C000:FOR I=0 TO 15:INK I,PEEK(&C7D0+I):NEXT:BORDER PEEK(&C7E0)` (`myapps/paint`) |

| | |
|---|---|
| **MUSIC** | reproduce canciones de Arkos Tracker 3 (`.MUS`, hechas con `make songs`), con tres barras de volumen. Está en su propio disquete, `music.dsk`, en B:. OPEN, SPACE reproduce y para. La música sigue detrás de otras vistas y programas, sin cortes, incluso mientras otro programa carga o dibuja: suena desde la interrupción. El bloque MUSIC de Studio lo inicia detrás de un programa con una canción (`myapps/music`) |
| **MARKDOWN** | un editor de archivos Markdown (`.MD`, hasta 6K), como NOTEPAD. ^P muestra la página: títulos `#`, **negrita**, *cursiva*, `código`, listas, citas, líneas, bloques de código y enlaces subrayados, sin las marcas. Las líneas seguidas forman un párrafo (dos espacios al final cortan la línea). Las flechas desplazan la página, RETURN o ^P vuelven al texto. Está en su propio disquete, `markdown.dsk`, con un `README.MD`. Menú EDIT: COPY ^C, CUT ^X, PASTE ^V, SELECT ALL ^A; SHIFT y las flechas seleccionan. El portapapeles (hasta 1K) es común a todos los programas (`myapps/markdown`) |
| **DIARY** | los días y lo que hay en ellos: el mes en modo 1, una semana por fila desde el lunes, el día elegido invertido y los días con notas marcados `*`, y debajo las notas del día elegido (hasta 10, de 40 caracteres). Las flechas mueven un día y una semana, `<` y `>` un mes, TAB va a las notas (UP/DOWN eligen una, RETURN la cambia, DEL la borra). ^A añade una nota, ^T va a hoy. Con una tarjeta M4 empieza en hoy; sin ella en el último día con una nota. Las notas se guardan en `DIARY.VDB` en la unidad desde la que se inició (^W la otra unidad), creado la primera vez, mediante `DB.SYS` (`myapps/diary`) |
| **SHOW** | las imágenes `.SCR` de una unidad una tras otra, en modo 0, con sus propios colores (los de PAINT): la siguiente cada cinco segundos, → y ← a mano, SPACE para y sigue, ^W la otra unidad (A:, B:, DISK). Hasta 48 imágenes (`myapps/show`) |
| **SYSINFO** | muestra las versiones de V-OS y de la API, la memoria (toda y la libre para programas) y el espacio libre en los disquetes de A: y B: (^R otra vez). Un ejemplo de `VOS_EXT` (`myapps/sysinfo`) |
| **TELNET** | texto por la red con la tarjeta M4: ^O CONNECT pide `HOST:PUERTO` (un nombre como `bbs.example.org`, o números como `192.168.1.10`; sin puerto, 23) y se conecta por TCP. Lo que llega se escribe en la ventana, que se desplaza hacia arriba al final; los colores y otros códigos ANSI se omiten, el griego en ISO 8859-7 se ve tal cual, y las opciones telnet que pide el otro extremo se rechazan. La línea escrita abajo se envía con RETURN. ^D CLOSE termina la conexión, como puede hacerlo el otro extremo; la primera línea muestra el estado (CONNECTED, NO CONNECTION...). Sin tarjeta M4: NO M4 BOARD (`myapps/telnet`) |
| **WEBGET** | descarga archivos de la web con la tarjeta M4: GET A FILE (^G) pide una dirección (HTTP, p. ej. `example.com/index.html`) y un nombre, y el archivo (hasta 16K) se guarda en la unidad que muestra (OTHER DRIVE). También muestra el estado de la red. Sin tarjeta M4: "NO M4 BOARD" (`myapps/webget`) |
| **READER** | lee páginas Markdown (`.MD`, hasta 6K) del disquete o de la web con la tarjeta M4, mostradas como la vista previa de MARKDOWN. TAB elige el enlace siguiente (invertido), RETURN lo abre, DEL vuelve a la página anterior (guarda 4). Un enlace sin `http://` se abre junto a la página, en el disquete o en el mismo sitio. OPEN A FILE y OPEN AN ADDRESS (^U, HTTP, p. ej. `example.com/README.md`). Las direcciones `https://` no se abren. Está en su propio disquete, `reader.dsk`, con `INDEX.MD` y `ABOUT.MD` (`myapps/reader`) |
| **STREETS** | un callejero en una base de datos de V-OS: escribe las primeras letras de una calle y ve las calles que empiezan así, con su zona y código postal; DEL borra una letra. ^A añade una calle, ^D borra la primera de la lista. Las calles están en `STREETS.VDB` en su disco, `streets.dsk`, en B:; la primera vez lo crea con calles de Atenas. Un ejemplo de la API de bases de datos (`myapps/streets`) La búsqueda no distingue mayúsculas, minúsculas ni acentos. |

| | |
|---|---|
| **DATABASE** | bases de datos, como un pequeño Access: ^N una base nueva (`NOMBRE.VDB`), ^O abrir una, ^T una tabla nueva (su nombre, y campos: un nombre, TEXTO hasta 40 caracteres o NÚMERO 0–65535, y si se ordena por el primero). La lista muestra un registro por línea y los campos en columnas; las flechas eligen y desplazan, TAB pasa a la tabla siguiente, y si la tabla tiene clave, lo que escribes va a los registros que empiezan así (sin mayúsculas ni acentos). RETURN abre el formulario: un campo por línea, RETURN lo cambia, ^S guarda, ^B vuelve a la lista. ^A un registro nuevo, ^D borrar, ^W la otra unidad, ^P imprime la tabla como la muestra la lista, todos los registros. PRINT LETTERS ^L imprime un `.TXT` de la unidad (una carta escrita en NOTEPAD) una vez por cada registro de la tabla, una página cada una: el nombre de un campo entre llaves, `{NAME}`, se vuelve lo que el registro tiene en él (sin mayúsculas; un nombre desconocido queda como está). IMPORT CSV añade un registro por cada línea de un `.CSV` a la tabla en uso: los campos en orden, separados por comas o por punto y coma, los textos cortados a su longitud, los números según sus cifras; una primera línea que empieza con el nombre del primer campo (los encabezados) se omite. También abre los `.VDB` de STREETS (`myapps/database`) |
| **SHEET** | una hoja de cálculo: 16 columnas (A–P) y 64 filas, 5 columnas y 19 filas en la pantalla. Una celda tiene, hasta 40 caracteres, un número (`12`, `-3.5`; el separador decimal es un punto), texto, o una fórmula después de `=`: `+ - * /`, paréntesis, celdas (`B3`), SUM, AVG, MIN, MAX y COUNT de celdas y rangos (`=SUM(B2:B9)`), ABS, INT, ROUND(x, 0–2). Los números tienen dos decimales, hasta ±21474836.46. Las flechas mueven el cursor, RETURN cambia la celda, al escribir empieza un texto nuevo, DEL (o CLR) la vacía; después de cada cambio se vuelven a calcular todas las fórmulas. La primera línea muestra el texto de la celda. Una celda con error muestra `#ERR`, un número que no cabe `########`. ^N nueva, ^O abrir, ^S guardar, ^A guardar como (`NOMBRE.VSH`, un archivo de texto, una línea por celda: `B3 =B1*2`), ^W la otra unidad, ^P imprime la hoja como la muestra la página, las columnas A–H y luego I–P, hasta la última fila usada. ^G CHART: un rango de una columna o de una fila (`B2:B9`, hasta 16 celdas; al principio la columna del cursor hasta su último número) como barras en modo 0, cada una tan alta como lo es la celda respecto a la mayor, con los textos de la columna A (o de la fila 1) debajo; ^S guarda el gráfico como `.SCR` (las barras, sin los textos), que abren PAINT y el BASIC, cualquier otra tecla vuelve. OPEN CSV lee un `.CSV` (de una hoja de cálculo de PC, o de una base de datos): una línea por fila, los campos desde la columna A, separados por comas o por punto y coma (la primera línea muestra cuál); los campos entre comillas pueden contenerlos, y con punto y coma `1,5` es el número 1.5. Al guardarlo, se convierte en un `.VSH` del mismo nombre. Las fórmulas las calcula `CALC.SYS` en A:, que cualquier programa puede usar (`VX_CALC`). Está en su propio disco, `sheet.dsk`, con el ejemplo `BUDGET.VSH` (`myapps/sheet`) |
| **PONG** | una pelota y una pala con las flechas, puntuación y vidas |
| **CATCH** | atrapar las estrellas que caen |
| **STARCAT** | un gato con disfraces que camina, una estrella, una melodía a partir de una lista |
| **COUNTER** | un contador con botones +1, −1 y 0, en modo 1 |
| **FLAP** | un pájaro que bate las alas (disfraces), y una estrella que cambia de disfraz cada segundo |
| **TICKER** | cuenta segundos y sigue en segundo plano |

- Los programas de bloques son archivos `.VBN`. Tienen un menú PROGRAM > STOP (CONTROL+Q).
- Los que tienen también un `.VBS` (CALC, MINES, FLAP, TICKER) se abren en Studio para verlos o cambiarlos.
- `VBRUN.SYS` no es un programa: es el "reproductor" que necesita Studio.

## Imprimir

NOTEPAD, SHEET y DATABASE imprimen con ^P (PRINT en su menú) en una impresora conectada al puerto de impresora del CPC (Centronics), como la Amstrad DMP2000 o una compatible con Epson.

- **Siete bits:** el puerto del CPC envía 7 bits, así que las letras por encima de 127 salen como letras simples: los acentos se omiten (É → E, ñ → n, ß → s) y el griego se escribe en Greeklish, letra por letra (Α → A, Θ → 8, Ξ → 3, Ψ → 4, Ω → W).
- **Las páginas:** cada página empieza con un encabezado, el nombre del archivo y el número de la página (`NOTES.TXT PAGINA 2`), y tras 60 líneas empieza sola una página nueva.
- **El final:** cada impresión termina con una página nueva.
- **Sin impresora:** si la impresora sigue ocupada dos segundos (ninguna conectada, fuera de línea, sin papel), aparece LA IMPRESORA NO ESTÁ LISTA y la impresión se detiene.
- Los programas imprimen con la llamada `VX_PRINT` (capítulo 7).

# 4. Idiomas

V-OS habla seis idiomas. El inglés viene incorporado. Los otros cinco son **paquetes de idioma**, archivos del disquete que traen tres cosas: todos los textos del sistema, el teclado y las letras (la fuente) del idioma.

| Idioma | Archivo | Teclado |
|---|---|---|
| English | incorporado (`ENGLISH.MSG`) | el inglés del CPC |
| Ελληνικά | `GREEK.LNG` | griego, con acentos |
| Français | `FRENCH.LNG` | AZERTY, cifras con SHIFT, teclas muertas ^ y ¨ |
| Español | `SPANISH.LNG` | Ñ en la tecla `:`, teclas muertas ´ ¨ ` ^, y ¡ ¿ |
| Deutsch | `GERMAN.LNG` | QWERTZ, Ü Ö Ä ß, teclas muertas ´ y ` |
| Italiano | `ITALIAN.LNG` | è é ò à ù ì en sus propias teclas |

## Cambiar de idioma

- **De forma permanente:** en Ajustes (F4), la línea IDIOMA recorre todos los idiomas. Cada uno se carga al momento desde A:, y toda la pantalla se vuelve a dibujar en él. Al guardar (^S) se mantiene también para el siguiente arranque.
- **Rápidamente:** CONTROL+L (VER > CAMBIAR DE IDIOMA) cambia entre el inglés y el idioma del paquete cargado.
- **Solo el teclado:** CONTROL+SHIFT cambia el teclado. La barra de menús muestra a la derecha cuál está activo (EN, ΕΛ, FR, ...). Con TECLADOS de Setup en EN + IDIOMA va entre el inglés y el del paquete; en LOS SEIS pasa por todos: EN, el del paquete, los otros cuatro, y otra vez EN. Así puedes escribir francés con el teclado francés mientras el sistema está en griego. Cada paquete contiene los teclados de todos.

Los símbolos `@ # [ ] { } \ |` no están en los teclados de los paquetes, porque el CPC no tiene AltGr. Escríbalos con el teclado inglés.

## Los textos de los programas

Los programas tienen sus propios textos en **dos idiomas**: inglés y, de forma opcional, uno más (hoy, griego). Un programa aparece en su segundo idioma solo cuando el paquete está en ese idioma. Si no, aparece en inglés. Así, con el francés, Studio aparece en inglés. Los programas de Studio, en cambio, pueden tener sus textos en los seis idiomas (capítulo 5): CALC se llama «CALCULADORA» en español.

## Un alfabeto para todos

Todos los idiomas escriben con el mismo juego de caracteres, el **juego de V-OS**, y cada paquete tiene todas sus letras. Así un texto o una base puede mezclar griego, francés, alemán, español e italiano, y todo se ve bien sea cual sea el idioma activo: en Setup, LANGUAGE muestra ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL con sus propias letras.

Los códigos 128 a 255 son los del griego en ISO 8859-7, tal cual, y los códigos que deja libres llevan À Á Â Ä Ç È É Ê Ë Ì Í Î Ï Ñ Ò Ó Ô Ö Ù Ú Û Ü, sus minúsculas, ß y ¡ ¿. El acento ´ se escribe como el tonos griego ( ΄ ) y µ como µ. No hay ÿ, Œ ni œ. La tabla está en `tools/voscs.py`. Un archivo de texto griego de un PC (ISO 8859-7) se lee tal cual; uno francés o alemán en Latin-1 hay que convertirlo.

Con el teclado inglés, SHIFT y ^ da £.

Los archivos en **UTF-8**, de un PC o de la web, los abren NOTEPAD y READER, que los pasan al juego de V-OS; lo que no tiene (chino, por ejemplo) se vuelve `?`. En el PC, `tools/voscv.py` convierte archivos desde y hacia V-OS (UTF-8, Latin-1 y otros) y dice qué es un archivo (`check`).

## Crear un idioma nuevo

Las fuentes de los idiomas están en `lang/<código>/`:

- `messages.txt`: los textos del sistema. La lista base la define `lang/en/messages.txt`, y lo que falte en un paquete aparece en inglés.
- `keymap.txt`: las teclas que difieren del teclado inglés.
- `pack.txt`: el código, el nombre, el juego de caracteres y la fuente.

`tools/mklng.py` los convierte en un `.LNG`, y el Makefile lo pone en el disquete. Todos los paquetes tienen la misma fuente, `lang/vos/font.txt`, y `charset vos`.

# 5. Studio: programas con bloques

STUDIO es un editor de bloques al estilo de Scratch que funciona en el propio CPC.

- Se hacen sprites y scripts con bloques.
- El programa se ejecuta al momento con CONTROL+R.
- Se convierte en un programa normal de V-OS (`.VBN`) que también se inicia desde Programas (F2).

## La pantalla

Studio usa una pantalla "dividida", con dos zonas:

- **Arriba, en modo 0 (16 colores):**
  - **a la izquierda, el escenario** a la mitad de tamaño
  - **a la derecha, el panel SPRITES:** los sprites con su color y su posición, el tamaño del seleccionado (W, H), el fondo y el modo del programa (p. ej. `STAGE 2 M1`), y el nombre del archivo
- **Abajo, en modo 1:**
  - **a la izquierda, la paleta** con los bloques de una categoría (`< EVENTS >`)
  - **a la derecha, el script**, una línea por bloque
  - **al final, la línea de estado**, con las teclas que funcionan donde se encuentra

**TAB** pasa del script a la paleta, luego al escenario, y vuelve al script.

## Menús

| STUDIO | | EDIT |
|---|---|---|
| NEW | ^N | DELETE LINE |
| OPEN (`.VBS`) | ^O | + SPRITE (hasta 6) |
| SAVE | ^S | − SPRITE |
| SAVE AS | | BACKGROUND (el color del escenario) |
| RUN | ^R | SCREEN MODE (0 o 1) |
| IN BACKGROUND | | TEXT LANGUAGE (^T) |
| BACK | ^Q | |

- **RUN (^R):** guarda `NOMBRE.VBS`, escribe `NOMBRE.VBN` y lo ejecuta. El programa se detiene desde su propio menú (PROGRAM > STOP, ^Q), y entonces Studio vuelve a arrancar con el mismo programa abierto.
- **IN BACKGROUND (en segundo plano):** pregunta si el programa sigue funcionando cuando está detrás de otro o detrás de una vista, o se pausa, que es lo predeterminado.
- **SCREEN MODE (modo de pantalla):**
  - **Modo 0:** 16 colores, 192 píxeles de ancho.
  - **Modo 1:** 4 colores (tintas 0–3), 384 píxeles de ancho, y más texto.

## La paleta

←/→ cambian la categoría, ↑/↓ eligen un bloque, y RETURN lo añade debajo de la línea del script. Un clic en un bloque de la paleta hace lo mismo.

Un bloque que abre (FOREVER, REPEAT, IF) entra junto con su END, y se borra junto con él. Un bloque con texto pide el texto al momento.

## El script

| Tecla | Qué hace |
|---|---|
| ↑/↓ | línea |
| ←/→ | hueco de la línea (los huecos son los valores dentro del bloque) |
| RETURN | sin hueco seleccionado: añade el bloque de la paleta. En un hueco de texto: cambia el texto. En un hueco de tecla: espera a que pulse la tecla |
| CLR | borra la línea |
| DEL | borra la última cifra de un número |
| ratón | clic en un hueco: lo selecciona. Segundo clic: cambia su clase |

El programa es una lista de líneas:

- Cada `WHEN …` empieza un script, para un sprite o para el escenario. El script llega hasta el siguiente `WHEN`.
- La sangría sale sola de los bloques que abren y cierran.
- Las líneas antes del primer `WHEN` se ignoran.
- Lo que quede abierto se cierra al final del script.

### Qué va en un hueco

La línea de estado siempre dice qué acepta el hueco.

**Hueco de valor:**

| Tecla | Valor |
|---|---|
| 0–9, − | número, un entero de −32768 a 32767 |
| A–H | variable. La misma letra otra vez: A1, A2 … A15, y vuelta a A |
| X, Y | la posición del sprite del script |
| R y cifras | RANDOM: un número de 1 a n |
| K y una tecla | KEY: 1 mientras se mantiene pulsada, si no 0 |
| T y una cifra | TOUCHING: 1 cuando el sprite toca el sprite n |
| Z | AT EDGE: 1 cuando el sprite está en el borde del escenario |
| P | BUTTON: el número del último botón pulsado |
| U | COSTUME: el número del disfraz del sprite (0: sin disfraz) |

**Otras clases de hueco:**

- **Variable:** A–H, y la misma letra otra vez para A1…A15.
- **Comparación:** `<`, `=`, `>`. SPACE da NONE ("ninguna"), y entonces IF solo comprueba que el primer valor no sea 0.
- **Sprite** (en los bloques `WHEN`): 0 el escenario, 1–6 un sprite.
- **Tecla** (en `WHEN KEY`): RETURN y luego la tecla.
- **Texto:** RETURN, y se escriben hasta 14 caracteres.

- **Grupo de variables** (SAVE/LOAD VARS): A–H pasa al siguiente grupo de ocho (A-H, A1-H1 … A15-H15), y SPACE da ALL.

Las letras se leen como teclas, así que también funcionan con el teclado griego.

## El escenario y los sprites

| Tecla | Qué hace |
|---|---|
| flechas | mueven el sprite seleccionado |
| 1–6 | seleccionan un sprite |
| C | color siguiente |
| W, H | ancho, alto |
| K | qué disfraz lleva (ninguno, 1, 2, …) |
| E | dibujar su disfraz (uno nuevo, si no tiene) |
| clic en el escenario | pone el sprite ahí |

Un sprite sin disfraz es un rectángulo de color.

### Disfraces

**E** abre el editor de disfraces en lugar del escenario. Tiene una cuadrícula de 16×16 con las 16 tintas debajo.

| Tecla | Qué hace |
|---|---|
| flechas | casilla |
| SPACE, RETURN | pinta la casilla |
| 0–9, A–F | tinta |
| . | transparente |
| E, TAB | terminado |
| clic | en una casilla: pinta. En la fila de tintas: selecciona una tinta |

Un programa tiene hasta 8 disfraces, compartidos por todos los sprites. El tamaño de un sprite con disfraz es 16×16. En el escenario, x se redondea a un byte entero (2 píxeles en modo 0, 4 en modo 1).

## Todos los bloques

En la columna "Bloque", los huecos aparecen como en el script.

### EVENTS

| Bloque | Qué hace |
|---|---|
| WHEN START *sprite* | el script empieza junto con el programa |
| WHEN KEY *tecla sprite* | empieza cuando se pulsa la tecla |
| WHEN CLICKED *sprite* | empieza con un clic en el sprite (o en el escenario) |
| WHEN I RECEIVE *n sprite* | empieza cuando se envía el mensaje n |
| SEND *n* | envía el mensaje n (n también puede ser una variable) |

> Un mensaje inicia **desde el principio** todos los scripts que lo esperan, aunque ya estuvieran en marcha. Por eso un script no debe enviarse un mensaje a sí mismo.

### CONTROL

| Bloque | Qué hace |
|---|---|
| FOREVER … END | repite para siempre |
| REPEAT *n* … END | n veces |
| IF *a comparación b* … [ELSE …] END | ejecuta las líneas solo si se cumple la comparación |
| WAIT *n* | espera n fotogramas (50 = un segundo) |
| WAIT UNTIL *a comparación b* | espera hasta que se cumpla la comparación |
| STOP | termina este script |

## MOTION

| Bloque | Qué hace |
|---|---|
| GO TO *x y* | el sprite va a x, y |
| CHANGE X *n*, CHANGE Y *n* | se desplaza n |
| SPEED *vx vy* | fija su velocidad |
| MOVE | un paso a su velocidad |
| BOUNCE | en el borde del escenario, invierte la velocidad |

## LOOKS/SOUND

| Bloque | Qué hace |
|---|---|
| COLOUR *n* | la tinta del rectángulo |
| COSTUME *n* | se pone el disfraz n (1–8. 0 no cambia nada) |
| NEXT COSTUME | el siguiente, y después del último otra vez el primero |
| SHOW, HIDE | |
| PRINT *valor* AT *x y* | escribe un número |
| STYLE *n* | el estilo de los PRINT y TEXT que siguen: 1 negrita, 2 cursiva, 4 subrayado, sumados (7: todos), 0 normal |
| NOTE *n* FOR *fotogramas* | toca una nota (número MIDI: 60 = do, 69 = la 440 Hz) y espera a que termine |
| MUSIC *"nombre"* | toca la canción *nombre*`.MUS` del disquete del programa, detrás de él, con el programa MUSIC (en A: o B:). Sigue después de que el programa termina, hasta el STOP de MUSIC (F6). Hace falta una ampliación de memoria; sin ella: "CIERRE PRIMERO OTRO PROGRAMA". Un nombre que no está (o vacío) para la música |

## VARIABLES

| Bloque | Qué hace |
|---|---|
| SET *variable* TO *valor* | |
| CHANGE *variable* BY *valor* | suma |
| DECREASE *variable* BY *valor* | resta |
| MULTIPLY *variable* BY *valor* | |
| DIVIDE *variable* BY *valor* | división entera |
| REMAINDER *variable* BY *valor* | el resto de la división |
| SAVE VARS *grupo* TO *"nombre"* | escribe las variables en un archivo de texto (véase más abajo) |
| LOAD VARS *grupo* FROM *"nombre"* | las vuelve a leer |

## WINDOWS

| Bloque | Qué hace |
|---|---|
| WINDOW *"título" ancho alto* | una ventana en el centro del escenario, como los diálogos de V-OS. Una ventana nueva ocupa el lugar de la anterior y de sus botones |
| BUTTON *"texto"* AT *x y* | un botón del sistema. Se pulsa con un clic o con TAB y RETURN. Los botones se numeran desde 0, en el orden en que aparecen |
| WHEN BUTTON *sprite* | empieza cuando se pulsa cualquier botón. El valor BUTTON (P) dice cuál |
| TEXT *"texto"* AT *x y* | escribe un texto |
| LABEL *botón* TO *"texto"* | cambia el texto de un botón |

Con una ventana abierta, TEXT, BUTTON y PRINT cuentan desde la esquina superior izquierda de su interior. La ventana se arrastra por su título, junto con sus botones.

## Variables

Hay 128 variables, compartidas por todos los scripts, llamadas A–H, A1–H1, ..., A15–H15. Al principio todas valen 0, y contienen enteros de −32768 a 32767.

### Guardar y recuperar (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` escribe las 128 en el archivo `GAME.TXT`. El `.TXT` se añade cuando el nombre no tiene tipo.
- Con un grupo (`A-H`, `A1-H1`, ...) escribe solo esas 8.
- El archivo es de texto, una línea por variable:

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` cambia las variables que menciona el archivo. Con un grupo, cambia solo las que también están en el grupo. Las líneas que no entiende se ignoran, y si falta el archivo no cambia nada.
- Así, un archivo puede recuperar todas las variables o solo un grupo de ocho, p. ej. las posiciones guardadas de un juego.

## Los textos en otros idiomas

Los textos de un programa (ventanas, botones, TEXT, LABEL) pueden tener una traducción en cada idioma de V-OS. El programa los muestra en el idioma en que está V-OS cuando arranca. Así CALC se llama «ΑΡΙΘΜΟΜΗΧΑΝΗ» en griego, «CALCULATOR» en inglés, «CALCULADORA» en español.

- **EDIT > TEXT LANGUAGE (^T):** pasa por los idiomas EN, EL, FR, ES, DE, IT y vuelve a los textos del propio programa. Bajo los sprites, junto al nombre, se ve el código del idioma (p. ej. `CALC ES`).
- En un idioma, las casillas de texto muestran y cambian los textos en ese idioma. Un texto sin traducción muestra el suyo, y al cambiarlo, eso pasa a ser su traducción. Un texto nuevo escrito en un idioma pasa a ser también el suyo.
- Las traducciones se guardan dentro del `.VBS` (y del `.VBN`), las seis juntas, unos 3K.
- En un `.vbt` se escriben con `LANG ES` seguido de líneas `"texto" "traducción"`.

## Archivos

- **`NOMBRE.VBS`:** el programa tal como se edita. OPEN lo abre, y se leen hasta 9K.

- **`NOMBRE.VBN`:** el programa listo para ejecutarse. RUN lo escribe, y también se inicia desde Programas (F2). Es el reproductor (`VBRUN.SYS`) junto con el programa. El reproductor compila los bloques a código Z80 al arrancar, así que los programas van rápido.

Un `.VBS` también se puede hacer a partir de texto en el PC, con `tools/mkvbs.py`. La sintaxis se describe al principio del archivo, y hay ejemplos en `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS). CALC y MINES tienen también traducciones (`LANG`).

## Límites

| | |
|---|---|
| Líneas (bloques) | 240 |
| Sprites | 6 |
| Scripts (`WHEN …`) | 16 |
| Variables | 128 |
| REPEAT en un programa | 64 (a partir de ahí comparten un contador) |
| Bloques abiertos anidados unos dentro de otros | 16 |
| Textos | 32, de hasta 14 caracteres cada uno, y en 6 idiomas |
| Disfraces | 8, de 16×16 |
| Botones | 64 |
| Código | al menos 4.000 bytes, incluso para el programa más grande. Lo que no cabe se ignora |

## Consejos

- **Scripts del escenario:** los bloques de movimiento y de apariencia no hacen nada, y X, Y, TOUCHING y AT EDGE dan 0.
- **TOUCHING:** solo se cumple cuando los dos sprites son visibles.
- **Sincronización:** los scripts funcionan "a la vez". Cada script cede su turno al final de cada bucle y en cada WAIT. Para que un script espere a otro, use una variable y `WAIT UNTIL` (véase MINES).
- **Ejemplos ya hechos:** CALC, MINES, FLAP y TICKER se abren en Studio (OPEN).

# 6. Bloques en el PC: el editor web y `vbc.py`

Además de Studio, que funciona en el CPC, existe también un editor de bloques para el PC. El lenguaje es el mismo, con algunos bloques más: listas, ruido, tamaño del sprite, texto de color.

## El editor

Abra `blocks/editor.html` en un navegador. Está basado en Blockly y viene en griego o en inglés.

- **Categorías:** eventos, control, movimiento, apariencia, texto, operadores, sensores, variables, listas y ventanas.
- **Sprites:** rectángulos o disfraces. Se definen en una tabla, y el escenario los muestra con los colores del CPC. Se pueden arrastrar con el ratón.
- **Archivos:** abre y guarda archivos `.vbp`. Son JSON con todo el programa, así que también se pueden escribir a mano.

## Compilar

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` escribe `build/NOMBRE.ASM`, con un comentario para cada bloque. Lo ensambla con RASM en `NOMBRE.VBN`, con una tabla de reubicación para que funcione junto a otros programas, y con `--dsk` lo pone en el disquete.

| Opción | Qué hace |
|---|---|
| `-o build` | carpeta de salida |
| `--asm-only` | solo el `.ASM` |
| `--dsk build/vos.dsk` | también en el disquete |

## El formato `.vbp`

La descripción completa está al principio de `blocks/vbc.py`: eventos, órdenes, valores, disfraces, icono. En resumen:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variables y listas:** tienen nombres, no letras. Las listas tienen una longitud máxima fija.
- **Icono:** `"icon"` da un icono para Programas (F2).
- **Un programa con muchas partes parecidas:** lo puede escribir un script de Python. Véase `tools/mkbricks.py`, que escribe `bricks.vbp` con 32 ladrillos.

## Ejemplos

En `blocks/examples/`:

| Archivo | Programa |
|---|---|
| `pong.vbp` | PONG |

| | |
|---|---|
| `catch.vbp` | CATCH |
| `starcat.vbp` | STARCAT: disfraces, una melodía a partir de una lista |
| `counter.vbp` | COUNTER: una ventana y botones |
| `bricks.vbp` | BRICKS (escrito por `tools/mkbricks.py`) |

Los archivos `*.vbt` son programas de Studio en forma de texto, para `tools/mkvbs.py`. Véase el capítulo 5.

## ¿Studio o editor web?

| | **Studio** | **editor web** |
|---|---|---|
| Dónde | en el CPC | en el PC |
| Archivo | `.VBS` | `.vbp` |
| Variables | 128, A–H15 | con nombres |
| Listas, ruido, tamaño del sprite | no | sí |
| SAVE/LOAD VARS | sí | no |
| Estilo del texto, música `.MUS` | sí | sí |
| El programa | necesita toda la memoria | pequeño, funciona junto a otros |

# 7. El SDK de V-OS: programas en ensamblador

Con este SDK se escriben programas propios en ensamblador Z80. Funcionan dentro de V-OS como DEMO o CLOCK: aparecen en Programas (F2), en el Escritorio y en el Commander, y tienen su propio menú, botones, diálogos, sonido y archivos. También funcionan junto a otros programas.

## Inicio rápido

```
make new-app NAME=hello
```

Esto crea la carpeta `myapps/hello/` a partir de `sdk/template/`. Contiene tres archivos:

- `hello.asm`: el programa. Las flechas mueven un recuadro, hay un botón BEEP, un contador de segundos y su propio menú (ABOUT, BACK) junto a VER y AYUDA.
- `texts.txt`: sus textos, en inglés y en griego.
- `icon.txt`: su icono, de 16×16, para Programas (F2).

```
make app NAME=hello
```

Esto compila `build/HELLO.BIN` y lo pone en el disquete de los programas, `build/apps.dsk`. `make run` lo pone en B:; en el Launcher, TAB muestra B:.

```
make run
```

Esto abre el disquete en Caprice32. HELLO está en Programas (F2), en la última página.

- Un `make` a secas compila todos los programas de `myapps/` y los pone en el disquete, junto con V-OS.
- El nombre tiene de 1 a 8 letras minúsculas o cifras y empieza por una letra. No puede ser el nombre de un programa de V-OS (p. ej. demo).
- Se necesitan RASM, Python 3 e iDSK, como para todo V-OS.
- En Caprice32 el programa se ejecuta a mano, desde Programas (F2).

Un programa más grande:

- Ponga más archivos `.asm` en su carpeta (p. ej. `art.asm` con sprites) e inclúyalos con `include "art.asm"`. El build los encuentra y vuelve a compilar el programa cuando cambian.
- Lo que ponga en la subcarpeta `disc/` (p. ej. `disc/LEVEL1.DAT`) va tal cual al disquete, para `VOS_LOAD`.
- Con una subcarpeta `owndisc/`, el programa tiene además su propio disquete, `build/<nombre>.dsk`, con el `.BIN` y los archivos de `owndisc/`. No va en `apps.dsk`. Se pone en B: y se ejecuta desde el Launcher (TAB, la otra unidad). La unidad desde la que se inició, para cargar de allí sus archivos, está en `VOS_SELF`: `db unidad`, y luego su nombre (11 bytes).
- **Un archivo para abrir (versión 14):** cuando RETURN en el Escritorio o el Commander abre un archivo en su programa, `VOS_OPEN` (`#B000`) contiene `db 1, unidad, nombre de 11 bytes` (0: ninguno). Léalo en `init`, con A = 0 y también con A = 2 (el programa ya estaba en marcha), y luego escriba 0 allí. Los tipos y sus programas están en la tabla `ft_open` del núcleo (`src/kernel/files_ext.asm`).
- Un programa con tabla de reubicación (como los del SDK) funciona junto a otros hasta 12K; uno mayor (hasta 16K) funciona solo sin ampliación de memoria, como uno sin tabla

(MARKDOWN, READER). Para lo demás, cargue archivos en el banco de datos.

## Cómo funciona un programa

Un programa es un `.BIN`, compilado para `VOS_APP_ADDR` (`#4000`). Empieza así:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
        jp init                 ; +0
        jp event                ; +3  HL -> el evento
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 el icono (opcional)
```

- **`init`** fija el modo, la paleta, el menú y los botones, activa `VE_FRAME` si hace falta, y lo dibuja todo.
  - ○ A = 0: un arranque nuevo.
  - ○ A = 2: el programa vuelve de una vista o de otro programa. Su estado es tal como lo dejó; vuelve a fijar lo anterior y dibuja.
- **`event`** se llama para cada evento y vuelve al momento. Un programa nunca espera por su cuenta (sin bucle de espera). Para movimiento o tiempo, `VOS_TICKS` con A = 1, y entonces llega un `VE_FRAME` 50 veces por segundo.
- **`redraw`** lo vuelve a dibujar todo, p. ej. cuando cambia el idioma o se cierra el accesorio de escritorio.
- **Salida:** `VOS_EXIT`, que vuelve a la vista que lo inició.

### El orden dentro de `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; los fotogramas
        push hl
        call VOS_UI_EVENT               ; menús, botones, CONTROL+letra
        pop hl
        or a : jr nz,command            ; A = la orden
        call VOS_MENU_OPEN : ret nz     ; menú abierto: las teclas son suyas
        ...                             ; sus propias teclas y clics
command: ...
        jp VOS_COMMAND                  ; lo que no es suyo (VER, AYUDA)
```

**Mientras hay un menú abierto, no dibuje.** Si no, dibujará encima de él. En `VE_FRAME`, actualice el estado, pero dibuje solo si `VOS_MENU_OPEN` da Z. Los diálogos (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) tienen su propio bucle, así que no llega ningún evento mientras están abiertos.

### Los eventos

`HL` → 8 bytes: `db type, code, char, mods; dw x, y`.

| type | qué | code / char / x, y |
|---|---|---|
| `VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2 | tecla | code = número de tecla (`VK_*`, fila del teclado·8 + bit), char = el carácter en el idioma del paquete |
| `VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4 | puntero | x, y en píxeles del modo |

| | | |
|---|---|---|
| VE_FRAME 8 | fotograma (con VOS_TICKS) | — |
| VE_LANG 7 | cambió el idioma del teclado | — |

### Tres clases de programa

- **Con tabla de reubicación (lo predeterminado del SDK):** `make app` lo compila dos veces, en `#4000` y en `#4100`. `tools/mkreloc.py` añade la tabla a partir de la diferencia.
  - Funciona junto a otros programas, en los 12K desde `#5000`, y continúa donde se detuvo (F6 muestra los programas en marcha).
  - Sus datos deben estar dentro de sus propios bytes (etiquetas, `ds`), nunca en direcciones fijas.
  - Si `mkreloc.py` da un error, en algún sitio se usa una dirección de una manera que no sobrevive a la reubicación. P. ej. `ld a,label & 255`, o solo el byte bajo de una dirección.
- **Sin tabla:** ocupa todo el banco (hasta 16K, código y datos), pero solo funciona solo, a menos que haya una ampliación de memoria. Necesita su propia regla en el Makefile (véase `STUDIO.BIN`).
- **Accesorio de escritorio:** hasta 4K, con tabla. Se abre con F5 encima de cualquier cosa, e `init` recibe A = 1. Véanse `apps/clock/clock.asm` y la descripción en `src/api/vos_api.inc`.

### Reglas

- **Registros:** las llamadas alteran todos los registros excepto IX. Los argumentos van en A, BC, DE, HL (IX para `VOS_RECT` y `VOS_BLIT`). Los resultados vuelven en A o HL. Si hay fallo, se activa el acarreo (carry) y A = número de error, que `VOS_ERROR` muestra.
- **Memoria:** no ponga otro banco en `#4000`, porque el programa desaparecerá bajo sus pies. Para 16K más está el banco de datos (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). Se comparte con las vistas y otros programas, y conserva sus datos solo mientras el programa está en marcha. Con una ampliación de memoria (API 21) cada programa recibe uno propio, que guarda lo que tiene hasta que el programa termina, corra lo que corra mientras tanto; `VX_DATA` dice cuál (A = 1: el suyo).
- **Coordenadas:** x en píxeles del modo (modo 0: 0–191, modo 1: 0–383, modo 2: 0–767), y de 0 a 258. La barra de menús ocupa y 0–11.
- **RASM:** las etiquetas no distinguen mayúsculas de minúsculas (`BOX` y `box` son lo mismo). El hexadecimal se escribe con `#` y no con `&`.

## Los formatos de datos

- **Barra de menús** (`VOS_SET_BAR`): `db count; dw menu, ...`. Los menús comunes VER y AYUDA vienen de `VOS_MENU_VIEW` y `VOS_MENU_HELP`.
- **Menú:** `db title, count` y para cada opción `db text, command, letter, 0`.
  - text: `VOS_SEPARATOR` para una línea
  - command: 1.., lo que devuelve `VOS_UI_EVENT`
  - letter: atajo CONTROL+letra, o 0
- **Botones** (`VOS_SET_WIDGETS`, luego `VOS_DRAW_WIDGETS`): `db count`, y para cada botón `dw x,`

`y; db ancho en caracteres, text, command, 0`.

- **Diálogo** (`VOS_DIALOG` → A = botón, 0 el primero, ESC da el último): `db title, text, número de botones (1-3), button, ...`. Para los botones, `VM_OK`, `VM_YES` y `VM_NO`.
- **Textos** (`VOS_TEXTS`): los genera `tools/mktexts.py` a partir de `texts.txt`, con líneas `NAME | English | Greek`. Las dos columnas se escriben en UTF-8 y se convierten en bytes del juego de caracteres de V-OS (`tools/voscs.py`), que contiene el griego y las letras latinas del francés, el español, el alemán y el italiano.
  - Da `T_NAME` (`VT_FIRST` = `#C0` en adelante) y la tabla `texts:`. Hasta 64 textos.
  - Un `|` sin espacios alrededor empieza una línea nueva en los diálogos.
  - Los textos van donde se pide un "texto" (menús, botones, diálogos). Con `VOS_MSG` se obtiene su dirección para `VOS_PRINT`.
- **Paleta** (`VOS_PALETTE`): 17 colores del firmware (0–26): 16 tintas y el borde. En modo 1 cuentan las tintas 0–3.
- **Rectángulo** (`VOS_RECT`, IX →): `dw x, y, w, h; db pen`.
- **Icono:** 16 líneas de 16 caracteres en `icon.txt`. Cada carácter es una tinta en hexadecimal (modo 0, paleta de Programas) o `.` para transparente. Programas lo lee del disco: mientras el programa corre, los 256 bytes después de "IC" quedan libres para búferes (PAINT guarda allí sus filas).
- **Nombres de archivo** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 bytes, `"NAME EXT"`. Unidad 0 = A:, 1 = B:.

## La API (versión 22)

| Llamada | Entrada → salida | Qué hace |
|---|---|---|
| `VOS_EXIT` | — | fin, vuelta a la vista que lo inició |
| `VOS_TICKS` | A = 1 / 0 (+ `VOS_PTR_KEYS`) | `VE_FRAME` 50 veces por segundo, o ninguno. Con `VOS_PTR_KEYS` (2) las flechas y COPY también mueven el puntero y hacen clic |
| `VOS_FRAMES` | → HL (y A bits 16–23) | fotogramas desde el arranque |
| `VOS_RANDOM` | → A | 0–255 |
| `VOS_COMMAND` | A = orden | las órdenes comunes de VER y AYUDA (6–19). Numere sus propias órdenes desde `VC_APP` (32), o un elemento de VER o AYUDA ejecuta una de las suyas |
| `VOS_ERROR` | A = error | diálogo de error |
| `VOS_MODE` | A = 0 / 1 / 2 | modo 0 (192×259, 16 tintas), 1 (384×259, 4 tintas) o 2 (768×259, 2 tintas, 96 columnas de texto; versión 25). En modo 2 los menús y diálogos se dibujan con la tinta 1 sobre la 0: tinta 0 blanca y 1 negra dan texto negro sobre papel blanco (NOTEPAD, MARKDOWN, READER, SHEET con ^M) |
| `VOS_PALETTE` | HL → 17 colores | |
| `VOS_CLS` | A = tinta | borra la pantalla |
| `VOS_PLOT` | HL = x, DE = y, A = tinta | un píxel |
| `VOS_RECT` | IX → rectángulo | rectángulo relleno |
| `VOS_COLOURS` | A = tinta del texto (+ `VOS_BOLD`, `VOS_ITALIC`, `VOS_UNDERLINE`), C = fondo | para `VOS_PRINT` y `VOS_CHAR`. Los estilos salen de la misma fuente, para todos los idiomas |

| | | |
|---|---|---|
| `VOS_PRINT` | HL = x, DE = y, BC → texto, 0 | |
| `VOS_CHAR` | HL = x, DE = y, A = carácter | |
| `VOS_MSG` | A = texto → HL | la dirección de un texto (`T_*` o uno del sistema) |
| `VOS_SET_BAR`, `VOS_DRAW_BAR` | HL → barra de menús | |
| `VOS_UI_EVENT` | (el evento actual) → A | orden de menú, botón o atajo, 0 si no hay |
| `VOS_DIALOG` | HL → diálogo → A | |
| `VOS_INPUT` | A = título, HL → texto, C = máximo → carry si se cancela | introducción de texto (necesita C + 2 bytes) |
| `VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS` | HL → botones | |
| `VOS_MENU_VIEW`, `VOS_MENU_HELP` | → HL | los menús comunes |
| `VOS_CATALOG` | A = unidad → A = archivos, HL → lista | 16 bytes cada uno: 11 el nombre, en +12 el tamaño en K. Con A = unidad + `VOS_CAT_EXT` y HL → `"EXT"`, solo los archivos con esa extensión |
| `VOS_FILE_PICK` | A = título → A = archivo, carry si se cancela | diálogo selector, después de `VOS_CATALOG` |
| `VOS_LOAD` | A = unidad, HL → nombre, DE = dónde → BC = longitud | |
| `VOS_SAVE` | A = unidad, HL → nombre, DE = desde, BC = longitud | archivo binario, que sustituye al anterior. Con A + `VOS_LOAD_IX` (#40, versión 20) su cabecera dice que se carga en IX, no en DE (un `.SCR`: IX = #C000) |
| `VOS_DELETE` | A = unidad, HL → nombre | |
| `VOS_TEXTS` | HL → tabla de textos | los textos del programa |
| `VOS_MENU_OPEN` | → NZ si hay un menú abierto | |
| `VOS_KEY_HELD` | A = tecla → NZ mientras está pulsada | para juegos |
| `VOS_BLIT` | IX → `dw x, y; db bytes, lines; dw source; db source stride` | imagen: pares (máscara, píxeles) |
| `VOS_SOUND` | A = canal 0–2, HL = periodo (bit 15: ruido), C = volumen 0–15, B = fotogramas | una nota; 440 Hz es el periodo 142 |
| `VOS_SPLIT` | HL → `dw y1, y2; db 8 colores` (0 = la desactiva) | pantalla mixta: modo 0 entre y1 e y2 |
| `VOS_RUN` | A = unidad, HL → nombre | ejecuta otro programa, y después este vuelve a arrancar |
| `VOS_KEEP` | → HL → 16 bytes | memoria que se mantiene entre programas |
| `VOS_BANK_GET`, `VOS_BANK_PUT` | HL = desde, DE = hasta, BC = bytes | copia desde y hacia el banco de datos |

| | | |
|---|---|---|
| VOS_BACKGROUND | A = 1 / 0 | sigue funcionando o se congela cuando está detrás de otro |
| VOS_DRAG | HL = x, DE = y, BC = w, A = h → carry: nuevos HL, DE | arrastrar una ventana por su título |
| VOS_FIT | igual → HL, DE | mete una ventana dentro de la pantalla |
| VOS_EXT | A = una llamada VX_*, BC, DE, HL según la llamada → A, HL, C, DE, acarreo | las llamadas después de la tabla (versión 15), abajo |

**Unidades:** 0 = A:, 1 = B:, 2 = DISK, la tarjeta SD de una M4 (versión 28), en la carpeta donde entró por última vez el Escritorio o el Commander. Las llamadas de archivos y VX_FREE funcionan allí como en un disquete (nombres 8.3; sin M4: acarreo, A = 35), y el catálogo tiene también las carpetas (en +13, 1). Las bases de datos (VX_DB) funcionan allí también (versión 29). Desde la versión 29, VOS_CAT_EXT conserva las carpetas, y VOS_FILE_PICK entra en una carpeta elegida (o `..`) y la lista como pidió el último VOS_CATALOG: lo que devuelve es siempre un archivo, en la carpeta actual. VX_DRIVE C = unidad → A = la siguiente (A: → B: → DISK con una M4 → A:), para una orden OTHER DRIVE; `src/api/vos_drive.inc` tiene `vos_next_drive` (A: ↔ B: en un V-OS anterior) y `vos_drive_text` (`A:`, `B:`, `DISK:`).

**Las llamadas de VOS_EXT:** VX_VERSION → A = API, HL = versión de V-OS (BCD). VX_FREE C = unidad → A = K libres. Desconocida: acarreo VX_CLIP_PUT HL = texto, BC = bytes → el portapapeles (hasta 1K). VX_CLIP_GET DE = dónde, BC = espacio → HL = tamaño (versión 16). VX_M4 HL → una orden para la tarjeta M4 (db tamaño, dw orden, datos), DE → la respuesta, BC = bytes; acarreo: no hay tarjeta M4 (versión 17); HL = 0: BC bytes del `sock_info` de la ROM de la M4 a DE, 16 por socket (su estado, su última orden, los bytes recibidos, una ip y un puerto; el socket 0 es el de C_NETHOSTIP, su ip el nombre buscado) (versión 32; TELNET espera así). VX_MEM → HL = la RAM en K, DE = los K libres para programas (versión 18). VX_BEHIND DE → db unidad, nombre de 11 bytes de un archivo: su programa (por tipo) arranca detrás de este, con `init` A = 3 y el archivo en VOS_OPEN (versión 19; MUSIC lo conoce). VX_DATA → A = 1 si el banco de datos es del programa (ampliación de memoria), 0 si es el banco 4 compartido (versión 21; NOTEPAD guarda así texto de hasta 16K). VX_DB C = una llamada DB_*, DE, HL → una base de datos (versión 22, abajo). VX_UTF8 HL → un texto, BC = bytes, E = 0 en la memoria del programa o VOS_DATA en su banco de datos: si está en UTF-8 (de un PC o de la web), pasa allí al juego de caracteres de V-OS → BC = bytes ahora, A = 1; A = 0: no era UTF-8 y queda como estaba (versión 23; así lo hacen NOTEPAD y READER). VX_CALC C = una llamada CALC_*, DE → un bloque, HL → texto → fórmulas y números (versión 24, más abajo). VX_LANG → HL = el código del idioma que muestra V-OS (`"EN"`, `"EL"`, `"FR"`...; L la primera letra) (versión 26; así muestra VBRUN las traducciones de los programas de Studio). VX_IRQ HL = una rutina del programa → el núcleo la llama una vez por imagen desde la interrupción, delante o detrás, con su banco puesto y las interrupciones desactivadas; 0 la detiene, igual que el fin del programa; acarreo: no para un accesorio ni para un programa solo en el banco 5 (versión 27; así toca MUSIC sus canciones, que ya no se paran mientras otro programa carga o dibuja). Debe ser corta, conservar IY, EXX y AF', escribir solo su propia memoria y el AY, y no llamar a nada de V-OS. VX_PRINT HL → un texto, BC = bytes, E = 0 en la memoria del programa o VOS_DATA en su banco de datos → a la impresora (el puerto Centronics del CPC, 7 bits), tal cual: 13, 10 terminan una línea, 12 empieza una página nueva; las letras por encima de 127 salen como ASCII simple (sin acentos, el griego en Greeklish: Θ 8, Ξ 3, Ψ 4); acarreo, A = 41: la impresora siguió ocupada dos segundos (ninguna, fuera de línea, sin papel) (versión 30; así imprimen NOTEPAD, SHEET y DATABASE con ^P). Tras 60 líneas empieza sola una página nueva; BC = 0 con HL → el nombre de 11 bytes de un archivo (como lo da VOS_CATALOG) empieza una impresión cuyas páginas llevan un encabezado, `NOTES.TXT PAGINA 2`; HL = 0: sin encabezados (versión 31). VX_VIEW HL → dw x, y, w, h: desde entonces el programa dibuja con (0, 0) en (x, y) y nada fuera de ese rectángulo (cortado a la

pantalla): VOS_CLS lo llena, VOS_RECT se corta a él, un punto, un carácter (cada uno de VOS_PRINT) o una imagen de VOS_BLIT se dibuja solo si está todo dentro; menús, diálogos y barra no cambian; HL = 0: ninguna; de nuevo tras VOS_MODE y en redraw (termina cuando otro programa pasa delante); carry: sin ampliación de memoria (versión 33; el primer paso hacia los programas en ventanas, docs/WINDOWS.md). VX_WINDOW HL → dw w, h: el programa en una ventana sobre el escritorio, su interior w × h (w múltiplo de 8) como vista y sus eventos del puntero con x, y desde él; llamarlo primero en init, cada vez (también con A = 2: así se redibujan las ventanas de detrás, con su init); su título la mueve, X lo termina, + o F7 la ponen a pantalla completa y de vuelta, un clic en otra la trae delante con su barra; modo 1, los colores del tema; carry: sin ampliación de memoria o sin WM.SYS en A: (pantalla completa como antes; versión 34). Desde la versión 35 VX_WINDOW escribe en HL el tamaño que tiene ahora su interior (pantalla completa: la pantalla bajo la barra), así init dispone lo que muestra para él; la esquina inferior derecha cambia el tamaño (y init se llama de nuevo con A = 2); con VOS_BACKGROUND el programa dibuja en su ventana también desde detrás, mientras ninguna la tape (VOS_CLS, PLOT, RECT, COLOURS, PRINT, CHAR, BLIT; no VOS_MSG). Ejemplos: apps/clock, myapps/sysinfo. Desde la versión 36 HL apunta a dw w, h, x0, y0: la ventana muestra la parte de la pantalla del programa desde x0, y0 (0, 0: como antes), así un programa hecho para toda la pantalla dibuja como antes, y sus widgets y eventos lo siguen (p. ej. dw 376, 232, 0, 12 en myapps/telnet); un programa con la versión 36 en su cabecera da siempre los cuatro. Si lo llama fuera de init, dibuja después su barra de nuevo (VOS_DRAW_BAR). Desde V-OS 0.49 una ventana de detrás que tiene una copia de sí misma (tomada mientras nada la tapaba) se copia de vuelta en su lugar: su init no se llama entonces, así que un programa no debe contar con que se le llame para dibujar estando detrás; uno que sigue en segundo plano (VOS_BACKGROUND) dibuja como antes mientras no está tapado. La ventana de archivos y los iconos de las unidades del escritorio también están entre las ventanas (0.50): hasta ocho ventanas en total. Desde la 0.51 VOS_PALETTE no hace nada mientras el programa está delante en una ventana (a pantalla completa funciona); la X de una ventana envía al programa CONTROL+Q, así que su BACK puede preguntar antes; un programa de detrás que nada tapa tiene también VOS_EXT (TELNET recibe ahí), y un VOS_EXT que no hizo nada mientras estaba tapado deja tus búferes como estaban: márcalos antes, como hace TELNET. Con w = 0 (V-OS 0.48) el programa sale de las ventanas: su ventana se va (carry), y pone su modo y su paleta y dibuja a pantalla completa, como para una página en modo 2; si lo pide de nuevo más tarde, la ventana vuelve donde estaba, del mismo tamaño. Con el tamaño devuelto un programa se dispone para su ventana: NOTEPAD, MARKDOWN, READER y SHEET (set_mode) toman w / 8 columnas y tantas filas como caben, de nuevo en cada cambio de tamaño, y salen de las ventanas para su página (^M).

## Bases de datos (versión 22)

VOS_EXT con A = VX_DB y C = una llamada DB_* trabaja con una base de datos: un archivo NOMBRE.VDB de páginas de 1K, tan grande como permita el disco (hasta 255K). Tiene hasta 8 tablas; una tabla tiene hasta 10 campos, texto (hasta 255 caracteres) o un número (2 bytes), y puede tener un campo clave (texto de hasta 64 caracteres, o un número) con un índice, para que FIND sea rápido. Un registro son sus campos uno tras otro; su id (rid) es un número de 16 bits. Hay una base abierta a la vez, para el programa que la abrió. Cada llamada escribe sus cambios en el disco antes de volver. El kernel carga el motor, DB.SYS, desde la unidad A: la primera vez.

| Llamada | Entrada → salida |
|---|---|
| DB_CREATE, DB_OPEN | DE → db unidad, nombre de 11 bytes ("STREETS VDB"); OPEN → A = 1: Latin-1 (de V-OS 0.15) |
| DB_CLOSE | |
| DB_TABLE | DE → nombre (8), campos, campo clave (0: ninguno), luego cada campo: nombre (8), tipo, longitud → A = tabla |
| DB_USE | E = la tabla que usan las llamadas de abajo |
| DB_INFO | E = tabla, HL → 124 bytes: su descripción → A = tablas |

| | |
|---|---|
| DB_ADD | HL → registro → DE = rid |
| DB_GET, DB_PUT | DE = rid, HL → el registro |
| DB_DEL | DE = rid |
| DB_FIND | HL → clave como en un registro (sitio para un registro), E = bytes que cuentan (0: todos) → DE = el primer registro con esa clave o después |
| DB_FIRST, DB_LAST, DB_NEXT, DB_PREV | HL → sitio para el registro, o 0 → DE = rid, por la clave; acarreo al final |
| DB_COUNT | → HL = registros |

Errores: acarreo y A (DB_ERR_*, y los errores del disco), que VOS_ERROR muestra. El texto se guarda tal como se escribe, en el juego de caracteres de V-OS, el mismo en todos los idiomas: una base puede tener griego, francés y alemán juntos. Solo una base de V-OS 0.15 con letras latinas (Latin-1) es distinta: DB_OPEN da entonces A = 1; muestra DB_ERR_ALPHABET con VOS_ERROR. El orden y FIND comparan bytes: en una clave DB_TEXT mayúsculas y minúsculas, y las acentuadas, son distintas. Un campo DB_ALPHA (tipo 2, desde la versión 23) es texto como DB_TEXT, pero como clave se ordena y se encuentra sin mayúsculas ni acentos: ΕΛΕΝΗ, Ελένη y ελένη, o Élodie y ELODIE, son lo mismo; el registro guarda el texto tal como se escribió. Ejemplos son STREETS (myapps/streets), un callejero, y DATABASE (myapps/database), un programa entero de bases con tablas, lista y formulario; tools/vdb.py muestra y comprueba un archivo .VDB en el PC.

## Fórmulas (versión 24)

VOS_EXT con A = VX_CALC y C = una llamada CALC_* calcula fórmulas, como una hoja de cálculo. El núcleo carga el motor, CALC.SYS, desde la unidad A: la primera vez. Un número es de 32 bits, en centésimas (1234 es 12.34), de -21474836.46 a 21474836.46. Una fórmula tiene + - * /, paréntesis, números (12, 3.5, .25; un tercer decimal redondea), celdas (A1 a Z255) y las funciones SUM, AVG, MIN, MAX, COUNT (de valores, celdas y rangos como B2:D9, separados por , o ;), ABS(x), INT(x) (hacia abajo, a un entero) y ROUND(x) o ROUND(x, 0–2), en mayúsculas o no. Los productos y los cocientes se redondean a la centésima.

DE → el bloque del programa, CB_SIZE (10) bytes: CB_RESULT (un número, 4 bytes), CB_COLS, CB_ROWS, CB_GRID (dw), CB_BANK, CB_AT. Las celdas son una cuadrícula del programa: CB_COLS × CB_ROWS números de 4 bytes, fila tras fila, en CB_GRID, en su banco de datos cuando CB_BANK = VOS_DATA. Una celda CALC_NONE no tiene número (vacía, o texto: 0 en + - * /, las funciones la omiten); CALC_BAD tiene un error.

| Llamada | Entrada → salida |
|---|---|
| CALC_EVAL | HL → fórmula, 0 (con = delante o no) → CB_RESULT y DE:HL (DE la parte alta) |
| CALC_NUMBER | HL → texto: ¿solo un número? → como CALC_EVAL |
| CALC_TEXT | CB_RESULT → texto en HL (sitio para 13 bytes: -12.5), A = su longitud |
| CALC_CELL | HL → texto: ¿una celda (b12) al principio? → D = columna, E = fila (desde 0), A = sus caracteres |

Errores: carry y A (CALC_ERR_*: 30 fórmula errónea, 31 división por cero, 32 número demasiado grande, 33 celda fuera de la cuadrícula, 34 una celda que usa tiene un error, 29 CALC.SYS no está en A:), que VOS_ERROR muestra; entonces CB_RESULT = CALC_BAD y CB_AT = dónde en el texto. El ejemplo es SHEET (myapps/sheet), una hoja de cálculo que guarda sus valores en su banco de datos y vuelve a calcular las fórmulas tras cada cambio.

Todos los detalles, p. ej. sobre el segundo plano, el accesorio y VOS_SPLIT, están en src/api/vos_api.inc. Como ejemplos, véanse apps/demo/demo.asm (modo 0, movimiento), apps/clock/clock.asm (accesorio) y apps/mixed/mixed.asm (pantalla mixta). Un juego completo con sprites, una pantalla leída de un archivo y sonido es myapps/vrouves/.

Un asistente de IA que escribe un programa debe leer AI.md (en inglés): las mismas reglas, más

cortas, con las trampas de RASM y del banco de datos.

## Pruebas

`make test-sdk` compila la plantilla como TEMPLATE.BIN y la abre desde Programas (F2) en Caprice32, con pulsaciones de teclas definidas por `src/kernel/test_script.asm` (script 15). Hace capturas de pantalla en `build/shots`: su pantalla, el recuadro después de moverse, su menú, el diálogo ABOUT y la vuelta a Programas.

Para su propia prueba automática, copie el script 15 y el objetivo `test-sdk` del Makefile.

# 8. Desarrollo de V-OS

Para quienes trabajan en el propio V-OS. Los programas *para* V-OS se describen en el SDK.

## Qué se necesita

- RASM, el ensamblador
- iDSK, para las imágenes de disquete
- Python 3, para las herramientas. `make docs` necesita además `markdown` y `pymupdf`.
- GNU make 4.3 o posterior
- para las pruebas, Caprice32 (snap `caprice32`)

## Compilación

| Orden | Qué hace |
|---|---|
| `make` | `build/vos.dsk` (A:): el núcleo, las vistas, los idiomas, DEMO, MIXED, STUDIO, CLOCK y las fuentes `.VBS`; `build/apps.dsk` (B:): los demás programas y `myapps/*`; y el disquete de cada programa con `owndisc/`. Si un archivo no cabe en un disquete real (180K), se detiene con un error |
| `make run` | abre el disquete en Caprice32 (`RUN"VOS"`) |
| `make shot` | arranca sin ventana y hace una captura de pantalla |
| `make docs` | el manual en PDF: `docs/V-OS.pdf` (griego) y `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it` |
| `make windows` | `build/V-OS-windows.zip`: `V-OS.exe` para Windows, con CPCEMU 3.0.2 (CPC 6128, 576K, ratón AMX, tarjeta M4), `vos.dsk` en A:, `apps.dsk` en B: y los disquetes de los programas al lado. V-OS arranca solo. Hace falta WSL y el zip portable de CPCEMU (`CPCEMU_ZIP=…`) |
| `make linux` | `build/V-OS-linux.tar.gz`: el script `V-OS` para Linux (x86_64), con CPCEMU 2.5 (la 3.0.2 no ha salido para Linux) y los mismos ajustes y disquetes. Hace falta SDL2 en el sistema y el tar.gz de CPCEMU para Linux (`CPCEMU_TGZ=…`) |
| `make songs` | las canciones de Arkos Tracker 3 como archivos `.MUS` en `myapps/music/owndisc` (`tools/mkmusic.py`; hace falta Arkos Tracker 3, `ARKOS=…`) |
| `make new-app NAME=x`, `make app NAME=x` | su propio programa (SDK) |
| `make clean` | borra `build/` |

## Estructura del repositorio

| Carpeta | Contenido |
|---|---|
| `src/vos.asm` | la raíz de la compilación: cargador y núcleo, escritura en el disquete |
| `src/boot/` | el cargador que AMSDOS ejecuta en `#4000`, detección de la ampliación de RAM |
| `src/kernel/` | vectores, interrupciones, sistema de archivos, vistas, API (`api.asm`), ajustes, extensión del núcleo (`ext.asm`), scripts de prueba (`test_script.asm`) |
| `src/drivers/` | Gate Array, CRTC, teclado, ratón AMX, FDC, AY |
| `src/gfx/` | gráficos overscan, texto, pantalla mixta |
| `src/ui/` | menús, botones, diálogos, introducción de texto, selector de archivos |
| `src/shell/` | las vistas como overlays: Escritorio, Programas, Commander, Ajustes |

| | |
|---|---|
| `src/api/vos_api.inc` | la API para los programas |
| `lang/` | los idiomas (textos, teclados, fuentes) |
| `apps/` | los programas en ensamblador: DEMO, MIXED, STUDIO, CLOCK, y sus iconos |
| `blocks/` | el editor web, `vbc.py`, `runtime.asm`, el reproductor de Studio (`player.asm`), ejemplos |
| `sdk/` | el SDK: guía y plantilla de programa |
| `tools/` | herramientas: fuentes, idiomas, textos, iconos, reubicación, `.VBS` a partir de texto, capturas de pantalla, PDF |
| `docs/` | este manual en griego (`manual/`) y en inglés, francés, español, alemán e italiano (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), los PDF, los requisitos (`REQUIREMENTS.md`) |

## La memoria, en resumen

| Zona | Qué contiene |
|---|---|
| `#0000-#3DFF` | el núcleo. En la página cero, después de los vectores de reinicio, la tabla de la API en `#0108` |
| hasta `#4000` | la pila del núcleo |
| `#4000-#7FFF` | la "ventana" de bancos |
| `#8000-#FFFF` | la pantalla overscan de 32K. En sus huecos, búferes del núcleo |

Los bancos que entran en la ventana:

| Banco | Qué contiene |
|---|---|
| 4 | búfer de archivos y de carga, y el banco de datos del programa en marcha |
| 5 | la vista en `#4000-#4FFF`, y desde `#5000` los programas con tabla de reubicación (o un programa sin tabla en todo el banco) |
| 6 | lo que tapan los menús y los diálogos, y desde `#7000` el accesorio de escritorio |
| 7 | el paquete de idioma, las listas del Commander, los textos ingleses, y desde `#7000` la extensión del núcleo |

Con una ampliación de memoria, sus bancos acogen programas. Los detalles, dirección por dirección, están en `docs/REQUIREMENTS.md` (§4.2 y en cada sección "Υλοποίηση M..." (implementación)).

## Convenciones

- **RASM:** el hexadecimal se escribe `#C000` y no `&C000`, porque RASM lee `&` como AND. Las etiquetas no distinguen mayúsculas de minúsculas.
- **Comentarios:** en inglés, breves, como en el código existente. Cada archivo empieza con un bloque que dice qué hace.
- **Documentación:** en griego. `MILESTONES.md` (inglés), `MILESTONES.el.md` (griego) y `docs/REQUIREMENTS.md` tienen una sección para cada hito. El manual se escribe en griego y se traduce a los otros cinco idiomas de V-OS.
- **Textos:** los textos del sistema están en `lang/*/messages.txt`, y los textos de los programas en `texts.txt` (para `tools/mktexts.py`).

## Pruebas

Las pruebas ejecutan V-OS en Caprice32 sin ventana y hacen capturas de pantalla en `build/shots/<retardo>/`.

**Cómo funcionan:**

- Caprice32 sin ventana no puede pulsar flechas, COPY ni teclas f. Por eso las pulsaciones las hace una compilación especial: un `-DTEST_SCRIPT=n` pone el script n en `src/kernel/test_script.asm`, una tabla de teclas por fotograma.
- Cada objetivo `make test-…` compila un disquete con su script y hace una captura en cada retardo de `…_SHOTS`.
- El retardo se mide en pulsaciones de "Z": unos 2,5–3,5 fotogramas cada una. La correspondencia no es fija, así que un momento se captura con dos o tres retardos a su alrededor.
- Una vez que V-OS escribe en el disquete, Caprice32 no se cierra solo, y la prueba termina por tiempo agotado. Esto es lo esperado en test-vars, test-costume y otras.
- Las capturas se miran a mano, p. ej. combinadas en una sola imagen con `ffmpeg … hstack/vstack`.

| Objetivo | Qué prueba |
|---|---|
| `test-input` | teclas, Studio: bloques, archivos, ejecución |
| `test-calc` | CALC con clics, luego `CALC.VBS` en Studio y su título en inglés y en francés (^T) |
| `test-costume` | disfraz en Studio, ejecución, reapertura |
| `test-acc` | el reloj como accesorio de escritorio |
| `test-multi`, `test-switch`, `test-expand`, `test-behind` | varios programas, F6, ampliación de memoria, segundo plano |
| `test-drag` | arrastrar ventanas |
| `test-langs`, `test-lang` | los idiomas, guardar el idioma |
| `test-mines`, `test-bricks` | los juegos |
| `test-vars` | las 128 variables, SAVE/LOAD VARS |
| `test-sdk` | la plantilla del SDK |
| `test-apps` | los dos disquetes tal como se entregan, `vos.dsk` (con las teclas del test) en A: y `apps.dsk` en B:: el Launcher muestra B: y PONG se ejecuta desde allí |
| `test-vrouves` | VROUVES desde su disquete en B:, por el Launcher (TAB): construir, la oleada 1, el dinero, HIGH SCORES |
| `test-notepad` | NOTEPAD: escribir, DEL, el diálogo de guardar, OPEN solo con archivos `.TXT`; luego ^M: la página (modo 2) |
| `test-notepad-big` | la ventana de texto de NOTEPAD en un emulador Z80 (`tools/notepad_check.py`): un archivo de 12K con su propio banco de datos, mover, escribir y SAVE, texto hasta 16K, y 4K en un 6128 |
| `test-data` | (576K) NOTEPAD con un texto de 15K hasta la línea 33, MARKDOWN mientras tanto, y NOTEPAD de vuelta al principio: las líneas desde su propio banco de datos |
| `test-paint` | PAINT: línea, rectángulo, relleno, los menús, y luego las capas (ocultar una), OTHER DRIVE y EXPORT .SCR a B: ("SAVED AS .SCR") |
| `test-paint-big` | PAINT en toda la pantalla (576K): líneas, la ventana movida por sus barras (un clic y un arrastre), su posición, guardada como `B:NONAME.SCR` y abierta de nuevo |
| `test-scr` | el EXPORT .SCR de PAINT (su código en un emulador Z80, `tools/scr_check.py`) cargado en BASIC desde un disco, con la paleta; se comprueba cada píxel de la pantalla |

| | |
|---|---|
| `test-about` | AYUDA > ACERCA DE V-OS |
| `test-music` | MUSIC desde `music.dsk` en B:: una canción suena y sigue detrás del Launcher |
| `test-markdown` | MARKDOWN desde `markdown.dsk` en B:: abre `README.MD`, la página con ^P, una pantalla más abajo, vuelta al texto; luego ^M: la página (modo 2) |
| `test-open` | abrir según el tipo en el Commander: `README.MD` en MARKDOWN desde B:, un `.TXT` en NOTEPAD desde A: |
| `test-sysinfo` | SYSINFO desde el Commander: versión, API, memoria, K libres en A: y B:, ^R, ^Q. Con `SYSINFO_RAM=576`, con una ampliación de 512K |
| `test-clip` | el portapapeles: seleccionar con SHIFT, ^C y ^V en NOTEPAD, ^A, ^X, ^V, y luego ^V en MARKDOWN |
| `test-webget` | a mano, en el CPCEMU de `make linux` (tarjeta M4; hacen falta una pantalla e internet): WEBGET descarga `example.com/index.html`. Cierre CPCEMU con F12 cuando aparezca SAVED, y se busca `INDEX.HTM` en el disquete |
| `test-reader` | READER desde `reader.dsk` en B: abre `INDEX.MD`, TAB elige un enlace, RETURN abre `ABOUT.MD`, DEL vuelve, y un enlace `https://` da un mensaje; luego ^M: la página (modo 2) |
| `test-db` | el motor de bases de datos en un emulador Z80 (`tools/db_check.py`): miles de llamadas al azar en cuatro tablas, comprobadas con un modelo, el archivo página a página (`tools/vdb.py`), hasta llenar el disco |
| `test-db-disc` | `DB.SYS` con el sistema de archivos del kernel en una imagen DSK (`tools/db_disc_check.py`) |
| `test-streets` | STREETS desde `streets.dsk` en B:: un `STREETS.VDB` nuevo con 24 calles, búsqueda, una calle nueva |
| `test-database-z80` | DATABASE en un emulador Z80, con la base en Python y la pantalla como texto: dos tablas, 47 registros desde el formulario, el orden sin mayúsculas ni acentos, la búsqueda, un cambio, un borrado, el desplazamiento |
| `test-database` | DATABASE en Caprice32: una base nueva en B:, una tabla, tres registros desde el formulario, la lista, la búsqueda, un cambio |
| `test-calc-sys` | el motor de fórmulas `CALC.SYS` en un emulador Z80 (`tools/calc_check.py`): fórmulas fijas y 40.000 aleatorias, números y textos, comparados con un modelo en Python |
| `test-sheet-z80` | SHEET en un emulador Z80 con el propio `CALC.SYS` y la pantalla como texto: valores, `#ERR`, celdas que se usan entre sí, desplazamiento, SAVE y OPEN, un archivo UTF-8, `BUDGET.VSH`, una hoja llena; luego ^M: la página (modo 2) |
| `test-sheet` | SHEET en Caprice32: números y una fórmula tecleados, `CALC.SYS` desde A:, la hoja guardada en B:; luego ^M: la página (modo 2) |
| `test-kbds` | TECLADOS: LOS SEIS en Setup, luego en NOTEPAD las teclas `:` y Z con cada teclado: `:z`, `:ζ`, `mw`, `ñz`, `öy`, `òz` |
| `test-charset`, `test-reader-utf8` | archivos UTF-8 en griego, francés, español, alemán e italiano en NOTEPAD y en READER |
| `test-utf8` | el código de `VX_UTF8` en un emulador Z80, frente a `tools/voscs.py` |
| `test-view` | `VX_VIEW`: VIEWT (`apps/viewt`) dibuja a través de una vista de 160 × 96, comprobada píxel a píxel (`tools/view_check.py`): la vista llena, un rectángulo cortado, 19 de 20 caracteres, el último píxel, una imagen dentro y otra fuera; con 128K: NO EXPANSION |
| `test-wm` | `VX_WINDOW` y `WM.SYS` (576K): WINA y WINB (`apps/wint`) en ventanas; siete capturas comprobadas (`tools/wm_check.py`): una delante de la otra, traída delante con un clic y su barra, un clic dentro desde su esquina, movida por su título, pantalla completa con F7 y de vuelta, cerradas con X hasta el Commander |

| | |
|---|---|
| `test-wm-apps` | CLOCK, SYSINFO y WINA en ventanas a la vez (576K); cinco capturas comprobadas (`tools/wm_apps_check.py`): las tres unas sobre otras, CLOCK traído delante con un clic y movido, SYSINFO delante y CLOCK en marcha detrás, sin tapar, SYSINFO reducido por su esquina |
| `test-wm-apps2` | DIARY, TELNET y CALC en ventanas (576K, API 36: la ventana muestra una parte de la pantalla del programa); cuatro capturas comprobadas (`tools/wm_apps2_check.py`): DIARY, TELNET encima, CALC con sus botones en su ventana, y 1 + 2 = 3 pulsados allí |
| `test-wm-notepad` | NOTEPAD en una ventana con un NOTES.TXT (576K): reducida por su esquina, sus columnas y filas cortadas a la ventana, la línea de estado debajo; la página (^M) a pantalla completa y la ventana de vuelta donde estaba (`tools/wm_notepad_check.py`) |
| `test-wm-pages` | MARKDOWN con INDEX.MD y su vista previa, READER (ajustado al ancho de la ventana) y SHEET (4 columnas, 18 filas) en ventanas (576K); la página de SHEET y la vuelta (`tools/wm_pages_check.py`) |
| `test-wm-copies` | copias de las ventanas (576K): WINB movida lejos de SYSINFO y WINA, sobre ellas y lejos otra vez; WINA y WINB escriben cuántas veces se han dibujado y siguen diciendo 001, con la imagen del escritorio debajo (`tools/wm_copies_check.py`) |
| `test-wm-desktop` | el escritorio entre las ventanas (576K, SETUP.CFG con la ventana de A:): WINA con la ventana y los iconos del escritorio detrás; el escritorio (F1) con WINA sobre su superficie; cada uno delante con un clic (`tools/wm_desktop_check.py`) |
| `test-wm-keys` | F5, F6, X y la paleta entre ventanas (576K): CLOCK como accesorio, WINA y WINB; la lista de programas (F6) y el accesorio sobre las ventanas, las ventanas otra vez al cerrarse; los rojos de WINA (P) dejan los colores como están; la X de WINA: ^Q, WINB delante (`tools/wm_keys_check.py`) |
| `test-styles` | STYLES.VBN (`blocks/examples/styles.vbt`) desde el Commander: normal, negrita, cursiva, subrayado y los tres, con el bloque STYLE |
| `test-music-block` | con 576K: TUNE.VBN (`blocks/examples/tune.vbt`) inicia MUSIC (desde B:) detrás de él con AGANAMEM.MUS; F6 muestra "> TUNE" y MUSIC, y RETURN trae MUSIC al frente: A:AGANAMEM, PLAYING. Con `MBLOCK_RAM=128`: "CIERRE PRIMERO OTRO PROGRAMA" |
| `test-amsdos`, `test-reboot`, `test-system` | archivos de V-OS en el BASIC, reinicio, disquete SYSTEM |

**Script nuevo:**

1. Añada un `IF TEST_SCRIPT == n` al principio de la cadena en `test_script.asm`, con `ELSE` antes del siguiente y un `ENDIF` más al final.
2. Añada un objetivo al Makefile, como `test-sdk`.

Cada tecla se escribe como `dw desde, hasta : db fila, bit` del teclado. El número de tecla es fila·8 + bit, p. ej. RETURN = 18 = fila 2, bit 2.

**Sin emulador:** el código de un programa también funciona en un Z80 dentro de Python (el paquete `z80`). Se carga el `.BIN` en `#4000` y se hace que las llamadas a la API vuelvan al momento (o hagan lo necesario, p. ej. archivos en un diccionario). Luego se llama a init y event y se lee la memoria. Es mucho más rápido que las capturas para errores en números, archivos y saltos. Las direcciones de las etiquetas salen de `rasm … -s -sl` (un archivo `.sym`).

## En cada hito

1. Código, y una prueba en Caprice32: nueva o existente, con capturas de pantalla.
2. Las pruebas a las que afecta el cambio.

3. Una sección en `MILESTONES.md` y `MILESTONES.el.md`, y una sección "Υλοποίηση M..." (implementación) en `docs/REQUIREMENTS.md`.
4. **Actualizar el manual en los seis idiomas** (`docs/manual/` en griego, `docs/manual/<código>/` para en, fr, es, de, it, y `sdk/README.md`, `sdk/README.<código>.md` si cambia la API), y luego `make docs` para todos los PDF.
5. Commit y push.

# 9. En un CPC real

V-OS solo se ha probado en emuladores: Caprice32, 1984 y CPCEMU. Un CPC real puede comportarse de otra manera en su unidad de disco, su teclado, su CRTC, su monitor o con un ratón AMX. Este capítulo es una lista de cosas que probar, y lo que cada una debe hacer. Si algo hace otra cosa, díganoslo. Incluso "todo funcionó" ayuda.

## Cómo llevar V-OS al CPC

La versión publicada tiene dos imágenes de disco, `vos.dsk` (A:) y `apps.dsk` (B:). Cada una cabe en un disquete de 3" (180K). Algunas formas de usarlas:

- **Un Gotek o un HxC** en lugar de una unidad: copie los archivos `.dsk` a su memoria USB.
- **Una unidad de 3,5" de PC** conectada al CPC como unidad B:: grabe las imágenes en disquetes (por ejemplo con SAMdisk o DiskImager en Windows).
- **Una tarjeta M4:** copie las imágenes a su tarjeta SD y móntelas desde su menú.

Después, en BASIC: `RUN"VOS`.

## Qué anotar primero

- El modelo (CPC 6128 o 6128 Plus) y, si lo sabe, el tipo de CRTC (0 a 4).
- La ampliación de memoria (ninguna, 64K, 512K...), las unidades, el ratón, una tarjeta M4.
- El monitor (CTM644 en color, GT65 verde, un televisor con un adaptador...).

## La lista

Cada línea: qué hacer, y qué debería ver.

**El arranque** - [ ] `RUN"VOS` desde A:. En unos segundos: el Escritorio, la barra de menús arriba, los iconos de las unidades a la derecha. Nada parpadea ni salta. - [ ] La imagen llena toda la pantalla (overscan, 384×272), sin nada cortado a los lados ni abajo. - [ ] AYUDA > ACERCA DE V-OS muestra el número de la versión publicada. - [ ] SYSINFO (en Programas, F2) muestra la memoria que tiene.

**La unidad de disco** - [ ] TAB en el Escritorio abre la ventana de A:, con sus archivos como iconos. Un segundo TAB: B:. - [ ] En el Commander (F3), copie un archivo de A: a B: (^C). Luego salga de V-OS (CONTROL+Q) y, en BASIC, `CAT` en B: lo muestra y `LOAD` lo lee. - [ ] Renombre (^N) y borre (^D) un archivo en B:. - [ ] FORMATEAR DISCO (DATA) en el menú ARCHIVO del Commander, en un disquete que no necesite. Luego `CAT` en BASIC: un disquete vacío. - [ ] También se lee un disquete en formato SYSTEM (el de CP/M). - [ ] Ajustes (F4): cambie el IDIOMA, ^S. Arranque de nuevo con `RUN"VOS`: V-OS arranca en ese idioma. - [ ] Sin disquete en B:, el Commander lo dice, y V-OS no se cuelga.

**El teclado** - [ ] Cada tecla en NOTEPAD: letras, dígitos, símbolos, SHIFT, CAPS LOCK, las flechas, DEL, CLR, COPY. - [ ] CONTROL+SHIFT cambia el idioma del teclado (EN, EΛ, ..., a la derecha de la barra de menús). Las letras griegas y acentuadas salen bien. - [ ] Una tecla mantenida se repite a la velocidad de RETARDO REPETICIÓN y VELOC. REPETICIÓN de Ajustes. - [ ] Las teclas de función: F1 Escritorio, F2 Programas, F3 Commander, F4 Ajustes, F6 los programas en marcha.

**El ratón AMX** - [ ] Ajustes: RATÓN AMX en SÍ. El puntero sigue al ratón a la misma velocidad en horizontal y en vertical, en modo 1 (el Escritorio) y en modo 0 (Programas). - [ ] Si salta o va en sentido contrario, pruebe el otro valor de LECTURA AMX. - [ ] Botón izquierdo: un clic; en el título de una ventana, la arrastra; en la esquina de abajo a la derecha, cambia su tamaño.

**La memoria** - [ ] Sin ampliación: NOTEPAD y PONG a la vez, F6 para pasar de uno a otro; STUDIO funciona solo. - [ ] Con ampliación: cuatro programas a la vez, cada uno donde se dejó. PAINT con una imagen grande. Una imagen en el Escritorio (Ajustes, IMAGEN DE FONDO).

**La pantalla** - [ ] PAINT y Programas en modo 0 (16 colores): los colores son correctos. - [ ] READER o NOTEPAD con una página en modo 2 (80 columnas): legible, también en un monitor en color. - [ ] La pantalla mixta de Studio (modo 0 arriba, modo 1 abajo): la línea de separación se queda quieta, nada parpadea. - [ ] El tema oscuro (Ajustes, TEMA).

**El sonido** - [ ] MUSIC toca una canción; sigue sonando mientras va al Escritorio y arranca otro programa. - [ ] Los juegos (VROUVES, BRICKS, PONG) tienen sus sonidos.

**La tarjeta M4** (si tiene una) - [ ] TAB en el Escritorio pasa de B: a DISK, la tarjeta SD, con sus carpetas. - [ ] NOTEPAD guarda un archivo en DISK, y lo vuelve a abrir. - [ ] La barra de menús muestra la hora de la M4; INFO DEL ARCHIVO de un archivo en DISK da su fecha. - [ ] WEBGET descarga un archivo; READER abre una página de la web. - [ ] TELNET se conecta a un servidor (una BBS) y muestra lo que envía.

**La velocidad** (cuánto tarda, con un reloj) - [ ] De `RUN"VOS` al Escritorio. - [ ] De F2 a Programas, y del icono de un programa al programa. - [ ] SHEET: BUDGET.VSH abierto y calculado.

## Cómo decírnoslo

Abra un issue en https://github.com/vaspervnp/V-OS-CPC/issues con:

- lo que anotó primero (el modelo, la memoria, el monitor...);
- las líneas que hicieron otra cosa, y qué hicieron;
- una foto de la pantalla, si algo se ve mal.